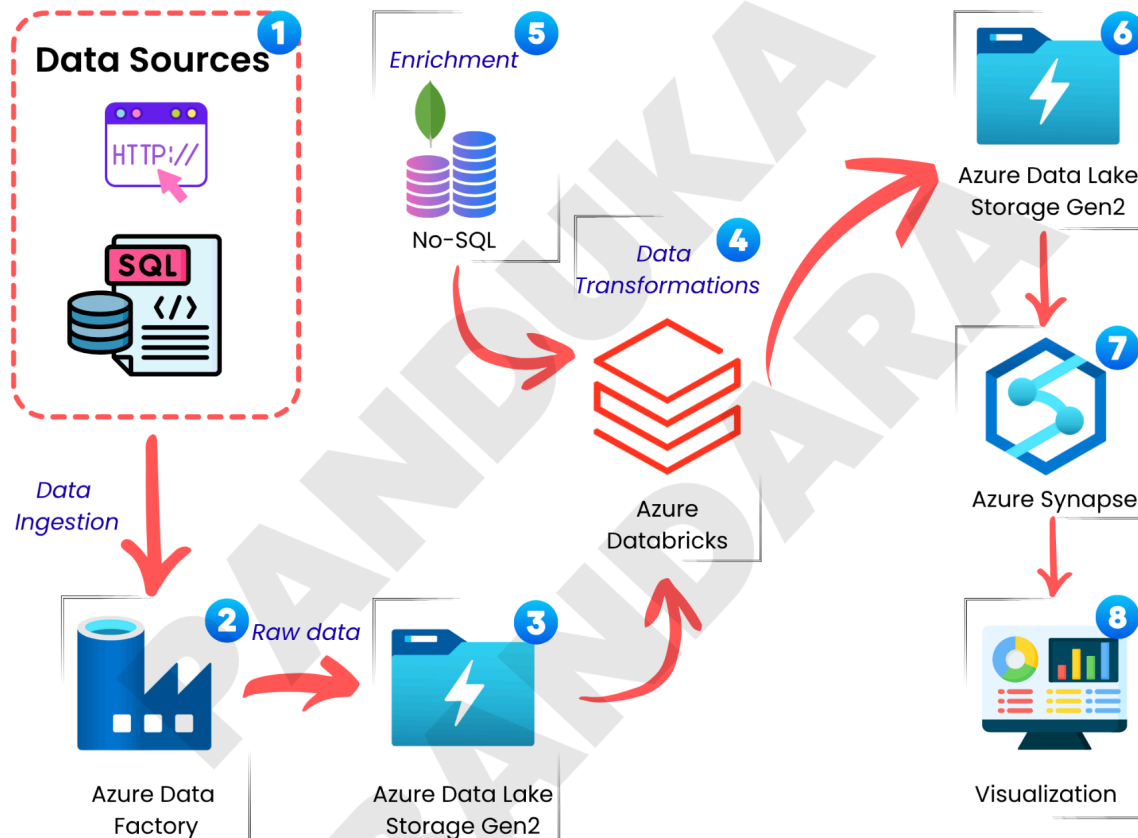


Project Architecture



Project Architecture
Panduka Bandara

The system is an end-to-end medallion architecture data pipeline implemented on the Microsoft Azure platform. The architecture ensures seamless data ingestion, transformation, storage, and visualization while maintaining scalability and integration across Azure services. The pipeline can be described in the following stages

1. Data Sources [1],[5]

The pipeline integrates data from a total of nine distinct tables. Specifically, seven tables are extracted from GitHub using the HTTP API. In addition, one table is retrieved from an Azure-hosted SQL Server database, and one table is obtained from Azure Cosmos DB configured with the MongoDB API

2. Data Ingestion with Azure Data Factory (ADF) [2]

Data ingestion is orchestrated using Azure Data Factory, which provides a serverless and scalable solution for automating data movement. ADF pipelines connect to the various sources,

extract data, and load them into the central storage layer.

3. Storage in Azure Data Lake Storage (ADLS) [3]

All ingested tables are stored in Azure Data Lake Storage (Gen2). ADLS is selected for its scalability, hierarchical namespace, and ability to efficiently store raw, curated, and transformed data for analytical workloads.

4. Data Transformation with Azure Databricks [4]

Azure Databricks is used to perform data preparation and processing. Key tasks include data cleaning and integration (joining tables). Used the Apache Spark engine to ensure distributed processing and efficient handling of large-scale datasets.

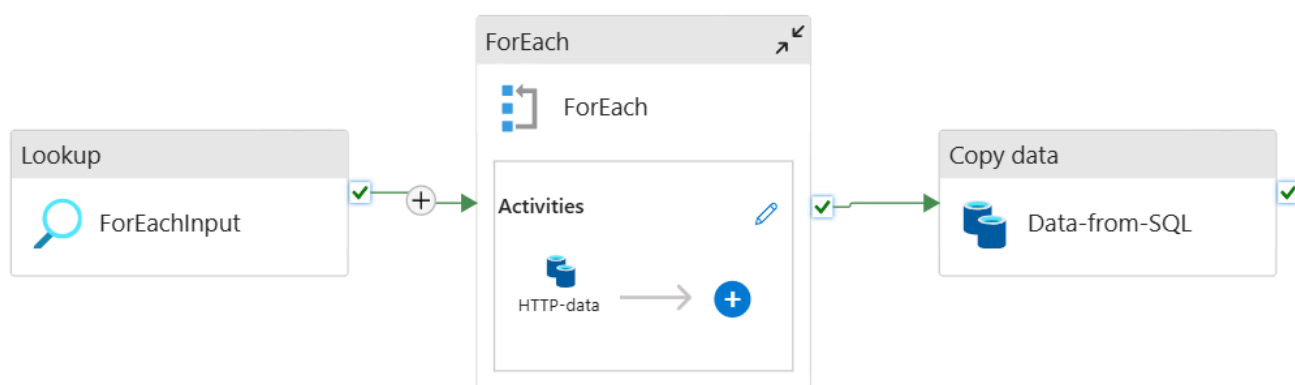
5. Analytics and Visualization with Azure Synapse Analytics [6],[7]

The transformed data is made available in Azure Synapse Analytics, which provides analytical capabilities. Additionally, Synapse is connected to another Azure Data Lake instance, which is used for its operations.

6. Data Consumption and Insights [8]

Finally, the curated data in Synapse is connected to visualization tools Power BI to generate dashboards and reports.

Layer	Data Type	Purpose	Azure Tools
Bronze	Raw, unprocessed	Keep original data, traceability	ADLS (raw), Data Factory
Silver	Cleaned, structured	Trusted, consistent, analytics-ready	Databricks, ADLS (curated)
Gold	Aggregated, business models	Reports, dashboards, KPIs	Synapse, Power BI, ADLS (gold)



Configurations - Summery

1. Databases

Summarized SQL Database Credentials

Database Name	ecommerce-sql-db-1
Server Name	ecommerce-sql-1.database.windows.net
User Name	admin123
Password	ecomdp#@@S

Summarized cosmosDB Credentials

Cluster Name	ecommerce-cosmos-db-1
Admin User Name	cosdmin123
Password	ecomdp#@@S
MongoDB Version	8.0

2. Links list

2.1 To get HTTP data

Name	httpGithubLinkedService
------	-------------------------

2.2 To get data from SQL server

Name	AzureSqlLinkedService
Type	Azure SQL Link
Configurations	As mentioned in SQL Database section

2.3 To send csv to ADLS gen2

Name	ADLSforCSV
Type	Azure Data Lake Storage Gen2`
Authentication Type	Account Key
Account Selection Method	From Azure Subscription
Storage Account Name	ecommercedatastorage9

2.4 To send sql to ADLS gen2

Name	ADLSforSQL
Type	Azure Data Lake Storage Gen2`
Authentication Type	Account Key
Account Selection Method	From Azure Subscription
Storage Account Name	ecommercedatastorage9

** Can use the same 2.3 but designwise its not good.

2.5 Get Data from HTTP

Name	httpGithubAdressListLink
Type	HTTP
Authentication Type	Anonymous
Base URL	raw.githubusercontent.com/PandukaBandara99/data-engineering/refs/heads/main/3-projects/Ecommerce-Data-Pipeline/ForEachInput.json

3. Datasets

3.1 Data from github [Source]

Name	github_csv_data
Data Store	HTTP

Format	Delimited text
Linked Service	httpGithubLinkedService
Base URL	https://raw.githubusercontent.com/PandukaBandara99/data-engineering/refs/heads/main/3-projects/Ecommerce-Data-Pipeline/Data-set/
First row as header	True
Import Schema	None
Add dynamic content	[Create Parameters : <code>csv_relative_url</code> -string] <code>@dataset().csv_relative_url</code>

3.2 As the sink for Data from github

Name	csv_from_http_to_sink
Data Store	Azure Data Lake Storage gen 2
Format	Delimited text
Linked Service	ADLSforCSV
File Path	ecom-data/bronze/
First Row as Header	True
Import Scheme	None
Dynamic content	[Create Parameters : <code>file_name</code> -string] <code>@dataset().file_name</code>

3.3 Data from Azure SQL [Source]

Name	data_from_azure_sql
Data Store	Azure SQL Database
Linked Service	AzureSqlLinkedService
Table Name	dbo.order_payments
First Row as Header	True
Import Scheme	None
Dynamic content	[Create Parameters : <code>file_name</code> -string] <code>@dataset().file_name</code>

3.4 As a sink for Data from Azure SQL [Source]

Name	from_sql_to_sink
Data Store	Azure Data Lake Storage gen 2
Format	Delimited text
Linked Service	ADLSforCSV
File Path	ecom-data/bronze/order_payments.csv
First Row as Header	True
Import Scheme	None

3.5 Get the CSV Adress List JSON [For Lookup - ForEachInput]

Name	get_HTTP_file_json
Data Store	HTTP
Format	JSON
Linked Service	httpGithubAdressListLink

4. Iteration & conditionals

4.1 For each block for HTTP data Ingestion

Name	ForEach
Sequential	True
Add dynamic content	<pre>[Create Parameters :ForEachInput -string] @pipeline().parameters.ForEachInput -Note that @pipeline().parameters-> Global variable\ - Values as below [{ "csv_relative_url": "customers_dataset.csv", "file_name": "customers_dataset.csv" }, { "csv_relative_url": "geolocation_dataset.csv", "file_name": "geolocation_dataset.csv" }, { "csv_relative_url": "order_items_dataset.csv", "file_name": "order_items_dataset.csv" }, { "csv_relative_url": "reviews_dataset.csv", "file_name": "reviews_dataset.csv" }, { "csv_relative_url": "orders_dataset.csv", "file_name": "orders_dataset.csv" }, { "csv_relative_url": "products_dataset.csv", "file_name": "products_dataset.csv" }, { "csv_relative_url": "sellers_dataset.csv", "file_name": "sellers_dataset.csv" }]</pre>

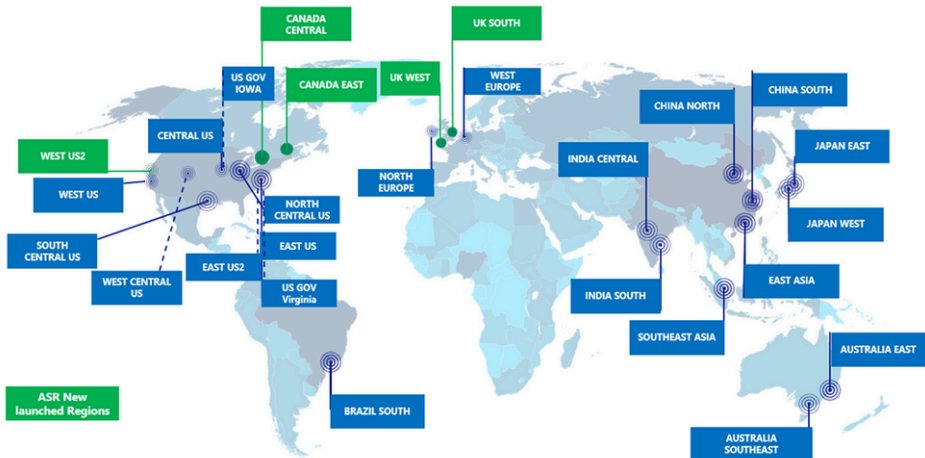
Build the Copy data block inside the for each. And add

- `@item().csv_relative_url` to the source
- `@item().file_name` to the sink.

The iteration will feed the value one by one.

Configurations - Detailed Notes

1. Create a resource group



[Holori - List of all Azure regions and Availability Zones](#)

Southeast Asia Central India South India Central US North Central US				
GEOGRAPHY	REGION	LOCATION	PERFORMANCE	LATENCY (MS)
Asia Pacific	Southeast Asia	Singapore	FAST	72 ms
India	Central India	Pune	ACCEPTABLE	104 ms
India	South India	Chennai	ACCEPTABLE	108 ms
United States	Central US	Iowa	SLOW	284 ms
United States	North Central US	Illinois	SLOW	299 ms

[Azure Latency Test](#)

Create the resource group. Since the project is for Sri Lanka, according to the latency test Southeast-asia is selected as the region.

[All services](#) > [Resource groups](#) >

Create a resource group ...

Basics Tags Review + create

Resource group - A container that holds related resources for an Azure solution. The resource group can include all the resources for the solution, or only those resources that you want to manage as a group. You decide how you want to allocate resources to resource groups based on what makes the most sense for your organization. [Learn more](#)

Subscription * ⓘ

Azure subscription 1

Resource group name * ⓘ

ecommerce-project

Region * ⓘ

(Asia Pacific) Southeast Asia

Settings for Resource Group

2. Create Datafactory

Create Data Factory ...

⚠ Changes on this step may reset later selections you have made. Review all options prior to deployment.

Basics

Git configuration

Networking

Advanced

Tags

Review + create

One-click to create data factory with sample pipeline and datasets. [Try it](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Azure subscription 1

Resource group *

ecommerce-project

Create new

Instance details

Name *

ecommerce-data-factory-1

Region *

Southeast Asia

Version *

V2

Settings for Azure Data Factory

3. SQL Database

Azure SQL is used.

How do you plan to use the service?

SQL Database Hyperscale: Low price, high scalability, and best feature set. [Learn more](#)

SQL

SQL databases

Best for modern cloud applications. Hyperscale and serverless options are available.

Resource type

Single database

Create

Show details

SQL

SQL managed instances

Best for most migrations to the cloud. Lift-and-shift ready.

Resource type

Single instance

Create

Show details

SQL

SQL virtual machines

Best for migrations and applications requiring OS-level access. Lift-and-shift ready.

Image

Create

Show details

☐ High availability

Azure SQL possible configurations

According to the Note 3.x , to satisfy our requirements Single SQL database is enough.

Create SQL Database ...

Microsoft

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name *

Server * ⓘ

[Create new](#)

Want to use SQL elastic pool? ⓘ ☐ Yes ☒ No

Workload environment ☒ Development ☐ Production

i Default settings provided for Development workloads. Configurations can be modified as needed.

Compute + storage * ⓘ

General Purpose - Serverless

Standard-series (Gen5), 1 vCore, 32 GB storage, zone redundant disabled

[Configure database](#)

Backup storage redundancy

Choose how your PITR and LTR backups are replicated. Geo restore or ability to recover from regional outage is only available when geo-redundant storage is selected.

Backup storage redundancy ⓘ ☒ Locally-redundant backup storage ☐ Zone-redundant backup storage ☐ Geo-redundant backup storage ☐ Geo-Zone-redundant backup storage

SQL Database Configurations

Use the same resource group and create a server.

Create SQL Database Server ...

Microsoft

Server details

Enter required settings for this server, including providing a name and location. This server will be created in the same subscription and resource group as your database.

Server name *	ecommerce-sql-1 ✓
	.database.windows.net
Location *	(Asia Pacific) Southeast Asia ▼

Authentication

 Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#) 

Select your preferred authentication methods for accessing this server. Create a server admin login and password to access your server with SQL authentication, select only Microsoft Entra authentication [Learn more](#)  using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [Learn more](#) , or select both SQL and Microsoft Entra authentication.

Authentication method	☐ Use Microsoft Entra-only authentication
	☐ Use both SQL and Microsoft Entra authentication
	☒ Use SQL authentication
Server admin login *	admin123 ✓
Password *	***** ✓
Confirm password *	***** ✓

OK

SQL Server Configurations

Summarized Database Credentials

Database Name	ecommerce-sql-db-1
Server Name	ecommerce-sql-1
User Name	admin123
Password	ecomdp#@@S

From the resource group, go to the SQL server and set up the public endpoint access. For the development & experimental purposes , Allow all IPs to access through the end point.

Enable Public Endpoint Access

1. Go to Azure Portal → SQL Server (not the database, but the parent server).
2. In the Networking tab
 - Public network access → Set to Enabled.

- If it's Disabled, switch it to Selected networks or All networks.

Configure Firewall Rules

By default, Azure SQL blocks all external connections need to allow IPs

- Under Firewall rules,
 - Add client IP (your current machine's IP).
 - To allow *all IPs*, set
 - Start IP: 0.0.0.0
 - End IP: 255.255.255.255

Public access Private access Connectivity

Public network access
Public Endpoints allow access to this resource through the internet using a public IP address. An application or resource that is granted access with the following network rules still requires proper authorization to access this resource. [Learn more](#)

Public network access

☐ Disable

☒ Selected networks

① Connections from the IP addresses configured in the Firewall rules section below will have access to this database. By default, no public IP addresses are allowed. [Learn more](#)

② Please save public network access value before adding new virtual networks.

Virtual networks
Allow virtual networks to connect to your resource using service endpoints. [Learn more](#)

+ Add a virtual network rule

Rule	Virtual network	Subnet	Address range	Endpoint status	Resource group	Subscription	State								
Firewall rules Allow certain public internet IP addresses to access your resource. Learn more + Add your client IPv4 address (112.134.179.25) + Add a firewall rule <table border="1"> <thead> <tr> <th>Rule name</th> <th>Start IPv4 address</th> <th>End IPv4 address</th> <th></th> </tr> </thead> <tbody> <tr> <td>allow-all</td> <td>0.0.0.0</td> <td>255.255.255.255</td> <td></td> </tr> </tbody> </table>								Rule name	Start IPv4 address	End IPv4 address		allow-all	0.0.0.0	255.255.255.255	
Rule name	Start IPv4 address	End IPv4 address													
allow-all	0.0.0.0	255.255.255.255													

Exceptions

☒ Allow Azure services and resources to access this server

Save **Discard**

Network Configurations for the SQL Server

To establish the connection through Google Collab ,

1. Must use pyodbc library [Not]
2. Must install ODBC Driver

```
pip install pyodbc
# To work ,Ensure you have ODBC Driver 18 for SQL Server installed on the system
# Add Microsoft repository
!curl https://packages.microsoft.com/keys/microsoft.asc | apt-key add -
!curl https://packages.microsoft.com/config/ubuntu/22.04/prod.list >
/etc/apt/sources.list.d/mssql-release.list

# Update packages
!apt-get update

# Install ODBC driver and unixODBC
!ACCEPT_EULA=Y apt-get install -y msodbcsql18 unixodbc-dev
```

4. Cosmos DB Mongo API

‘Azure Cosmos DB for MongoDB’ is used as document type data storage.

vCore cluster type is sufficient for the project scope.

Which type of resource?

Azure Cosmos DB for MongoDB offers two resource types with different architectures. Request unit (RU) database accounts and vCore clusters. [See documentation to](#)
To start, select the type to create a resource. The resource selection cannot be changed after creation.

vCore cluster (Recommended)

- Familiar architecture for new and existing MongoDB workloads
- Vector database capabilities to power Gen AI applications
- Instantaneous vertical autoscaling
- Ideal for long-running queries and complex aggregation pipelines
- [See documentation and supported features](#)

Create

Request unit (RU) database account

- For cloud-native MongoDB workloads with more point-reads and simple queries
- Instantaneous, granular autoscaling
- Multi-region write capabilities for mission critical applications
- Limitless horizontal scalability
- [See documentation and supported features](#)

Create

Encrypted connections

This cluster enforces encrypted connections using Transport Layer Security (TLS). For information on TLS version and certificates, refer to connecting with TLS/SSL. [Learn more](#)

Connectivity method * ⓘ

☒ Public access (allowed IP addresses)

☐ Private access

ⓘ Connections from the IP addresses configured in the Firewall rules below will have access to this cluster. By default, no public IP addresses are allowed. [Learn more](#)

Firewall rules

☐ Allow public access from Azure services and resources within Azure to this cluster

+ [Add current client IP address \(112.134.179.25\)](#) + [Add 0.0.0.0 - 255.255.255.255](#)

Rule Name	Start IP address	End IP address	
AllowAll_2025-9-8-10-25-58	0.0.0.0	255.255.255.255	

Cluster Name	ecommerce-cosmos-db-1
Admin User Name	cosdmin123
Password	ecomdp#@@S
MongoDB Version	8.0

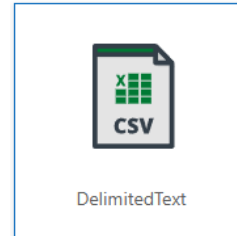
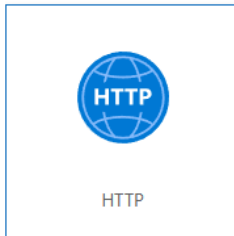
Settings for Cosmos DB setup

5. Build the pipeline

Go to the azure datafactory and Launch the studio. Then create the 'data-ingestion-pipeline' pipeline.

1. To get the data from **GitHub hosted csv s**

Add copy data block [name: data-from-github] . As the source use HTTP. and the format as delimited text.



Name as `github\_Data\_Csv`. Setup the **Linked Service** as below. [Additional Info in Notes 2.12/1]

Edit linked service

HTTP [Learn more](#)

Name *

github_link

Description

customers_dataset

Connect via integration runtime * ⓘ

AutoResolveIntegrationRuntime

Base URL *

<https://raw.githubusercontent.com/PandukaBandara99/data-engineering/refs/heads/main/3->

Information will be sent to the URL specified. Please ensure you trust the URL entered.

Server certificate validation ⓘ

☒ Enable ☐ Disable

Authentication type * ⓘ

Anonymous

Auth headers ⓘ

+ New

Annotations

+ New

> Parameters

> Advanced ⓘ

Important - A possible mistake !

1. **GitHub file URLs don't directly point to raw content by default**

- If you copy the URL from the browser address bar (e.g., <https://github.com/user/repo/blob/main/file.csv>), this is a **web page URL**, not the raw CSV file.

- ADF reads the URL and downloads the content it sees which is the **HTML page** that GitHub serves, not the CSV.
2. **ADF cannot parse HTML as CSV**, so you see lines like `<!DOCTYPE html>` in your output.

To fix it

You need to use the **raw file URL** from GitHub

1. Go to the file in GitHub.
2. Click **“Raw”** (top right of the file view).

It will take you to a URL like

<https://raw.githubusercontent.com/user/repo/main/file.csv>

3. Use this **raw URL** in your ADF **Linked Service / Dataset**.

So for the above connection use the base url as

<https://raw.githubusercontent.com/PandukaBandara99/data-engineering/refs/heads/main/3-projects/Ecommerce-Data-Pipeline/Data-set/>

And in the relative URL the file names can be used [to get all 8 csvs instead of creating 8 individual connections]

Do the configuration 6. And continue.

Connect the ADLS gen2 as the sink.



Azure Data Lake Storage
Gen2



DelimitedText

Set properties

Name

Linked service *

File path
 / /

First row as header ☒

Import schema
☒ From connection/store ☐ From sample file ☐ None

Advanced
[Open this dataset](#) for more advanced configuration with parameterization.

We need a new linked service for the sink.

New linked service

 Azure Data Lake Storage Gen2 [Learn more](#) 

Name *

ecom_datalake

Description

Connect via integration runtime * 

 AutoResolveIntegrationRuntime

Authentication type

Account key

Account selection method 

☒ From Azure subscription ☐ Enter manually

Azure subscription 

Azure subscription 1 (d9de67cc-364e-4a7c-a573-e6cbe2820de5)

Storage account name *

ecommercedatastorage9

Test connection 

☒ To linked service ☐ To file path

Note: After the debug run , it will create the customers_data.csv file in the data lake.

 ecom-data >  bronze

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

 Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 1 items

☐	Name	Last modified	Access tier	Blob type
☐	 [..]			
☒	 customers_...	9/11/2025, 2:39:05 PM	Hot (Inferred)	Block blob

Azure Blob Storage supports **three blob types**, each designed for different use cases:

Blob Type	Explanation	Best For	Example
Block Blob	Stores text or binary data in blocks. Most common type.	Large files, streaming, backups, images, documents.	Store sales_data.csv (100 MB), video files, PDFs.
Append Blob	Optimized for append-only operations. New data is always added to the end.	Logging, audit trails, telemetry.	Application writes logs continuously to applog_2025.txt .

Page Blob	Stores data in 512-byte pages for random read/write access.	Virtual hard drives (VHDs) for Azure VMs, databases requiring fast I/O.	Azure VM OS disk, database files.
------------------	--	---	-----------------------------------

Access tiers control **how data is stored** based on how often you access it. This helps balance **cost vs performance**.

Access Tier	Explanation	Best For	Example
Hot	Optimized for frequent access . Higher storage cost, lowest access cost.	Active data in use daily.	Transactional app data, frequently used reports.
Cool	Lower storage cost, higher access and retrieval cost. Minimum storage duration = 30 days.	Data that is infrequently accessed.	Monthly sales reports, semi-active backups.
Archive	Lowest storage cost, highest retrieval cost. Data must stay at least 180 days. Retrieval can take hours.	Long-term backups, compliance archives.	7-year financial records, legal data retention.
Premium	SSD-backed, low latency, high throughput. Available only for block blobs.	Performance-sensitive workloads.	Media streaming, big data analytics, IoT pipelines.

To automate the process of HTTP data ingestion , we use **for-each** Iteration & conditional block.

```
{
  "csv_relative_url" : "customers_dataset.csv",
  "file_name" : "customers_dataset.csv"
},
{
  "csv_relative_url" : "geolocation_dataset.csv",
  "file_name" : "geolocation_dataset.csv"
},
{
  "csv_relative_url" : "order_items_dataset.csv",
  "file_name" : "order_items_dataset.csv"
},
{
  "csv_relative_url" : "reviews_dataset.csv",
  "file_name" : "reviews_dataset.csv"
},
{
  "csv_relative_url" : "orders_dataset.csv",
  "file_name" : "orders_dataset.csv"
},
{
  "csv_relative_url" : "products_dataset.csv",
```

```

"file_name" : "products_dataset.csv"
},
{
  "csv_relative_url" : "sellers_dataset.csv",
  "file_name" : "sellers_dataset.csv"
}

```

Since we feed one element at once , make it sequential

6. Storage Accounts

We use azure data lake storage gen 2 (ADLS gen2) storage created under the same resource group.

Create a storage account ...

Basics Advanced Networking Data protection Encryption Tags Review + create

Azure Storage is a Microsoft-managed service providing cloud storage that is highly available, secure, durable, scalable, and redundant. Azure Storage includes Azure Blobs (objects), Azure Data Lake Storage Gen2, Azure Files, Azure Queues, and Azure Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#)

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription *

Resource group * [Create new](#)

Instance details

Storage account name *

Region * [Deploy to an Azure Extended Zone](#)

Preferred storage type

i This helps us provide relevant guidance. It doesn't restrict your storage to this resource type. [Learn more](#)

Performance * ☒ **Standard:** Recommended for most scenarios (general-purpose v2 account)
☐ **Premium:** Recommended for scenarios that require low latency.

Redundancy *

Need to enable **Hierarchical Namespace** otherwise it will be a just blob storage.

Hierarchical Namespace

Hierarchical namespace, complemented by Data Lake Storage Gen2 endpoint, enables file and directory semantics, accelerates big data analytics workloads, and enables access control lists (ACLs) [Learn more](#)

Enable hierarchical namespace ☒

Azure Storage Account Configurations - 2

Then add a container [name : ecom-data]

▼ Data storage

Containers

File shares

Queues

Tables

+ Add container

↑ Upload

🔍 Search containers by prefix

Showing all 1 items

☐

Name

☐

 \$logs

Name *

ecom-data ✓

Anonymous access level ⓘ

Private (no anonymous access) ▼

ⓘ The access level is set to private because anonymous access is disabled on this storage account.

^ Advanced

Encryption scope

Select from existing account scopes ▼

☐ Use this encryption scope for all blobs in the container

☐ Enable version-level immutability support ⓘ

ⓘ In order to enable version-level immutability support, your storage account must have versioning turned on.

Inside the container, Create three separate directories to store data at each.

+ Add Directory

↑ Upload

🔄 Refresh

🗑 Delete

📄 Copy

📂 Paste

🏷 Rename

🔒 Acquire lease

🔓 Break lease

 ecom-data

Authentication method: Access key (Switch to Microsoft Entra user account)

🔍 Search blobs by prefix (case-sensitive)

Showing all 3 items

☐

Name

☐

 bronze

☐

 gold

☐

 silver

Note :

redundancy options for Azure Storage Accounts

Redundancy Type	Explanation	Replication Scope	Use Cases / Notes
LRS – Locally Redundant Storage	Stores 3 copies of your data within a single data center in the primary region.	Within a single region	Cheapest option. Protects against hardware failures but not data center outages . Example: Dev/test data, temporary data.
ZRS – Zone-Redundant Storage	Stores data synchronously across 3 availability zones in the same region.	Across availability zones in the same region	Protects against data center failures . Good for production workloads requiring higher availability. Example: Critical app logs, VM disks.
GRS – Geo-Redundant Storage	Stores data LRS in primary region + asynchronous copy to another region (~hundreds of miles away).	Primary + secondary region	Protects against regional outages . Example: Production backups, disaster recovery.
GZRS – Geo-Zone-Redundant Storage	Combines ZRS + GRS . Stores data across zones in primary region + async copy to secondary region .	Zones in primary + secondary region	Maximum durability & availability. Example: Mission-critical applications needing both zone-level and regional disaster protection .
RA-GRS – Read-Access Geo-Redundant Storage	Like GRS, but allows read access from secondary region .	Primary + secondary region	Useful for disaster recovery scenarios where read operations are needed even during primary region downtime.
RA-GZRS – Read-Access Geo-Zone-Redundant Storage	Like GZRS, but secondary region is read-accessible .	Zones in primary + secondary region	Highest durability + read-access in case of region failure. Example: Global applications with strict uptime SLAs.

Hierarchical Namespace

Feature	Explanation	Example / Notes
Directory & File Structure	HNS allows you to create folders/directories like a traditional file system.	<code>/raw/sales/2025/january/data.csv</code> instead of flat <code>raw_sales_2025_january_data.csv</code> .
Atomic Operations	Operations like rename or delete on directories are atomic.	Renaming <code>/raw/sales/2025/january/</code> moves all files in that folder at once.
POSIX-like ACLs	Supports fine-grained access control at directory and file level.	Grant <code>read/write/execute</code> permissions for a specific user or group. Example: Only analysts can access <code>/processed/sales/</code> .
Scalability	Optimized for big data workloads with billions of files and directories.	Useful when using Spark, Databricks, or Synapse for processing large datasets.
Integration with Big Data Tools	HNS allows efficient directory-level operations for analytics.	Tools like Spark can read/write directories efficiently instead of individual files.

7. Lookup ForEachInput

General

 Append variable

 Delete

 Execute Pipeline

 Execute SSIS package

 Fail

 Get Metadata

 Lookup

8. DataBricks Environment

Create an Azure Databricks workspace ...

Basics Networking Encryption Security & compliance Tags Review + create

Project Details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *	Azure subscription 1
Resource group *	ecommerce-project

[Create new](#)

Instance Details

Workspace name *	ecommerce-spark-workspace
Region *	Southeast Asia
Pricing Tier *	Premium (+ Role-based access controls)
 We selected the recommended pricing tier for your workspace. You can change the tier based on your needs.	
Managed Resource Group name	ecommerce-databricks-resource-group

** Note that databricks need its own resource group

Basics Networking Encryption Security & compliance Tags Review + create

Deploy Azure Databricks workspace with Secure Cluster Connectivity (No Public IP) ☐ Yes ☒ No

Deploy Azure Databricks workspace in your own Virtual Network (VNet) ☐ Yes ☒ No

Inside the data bricks platform , first need to create a compute engine.

After that need to connect **Azure Datalake** to the databricks. For that need to create an app registration to authenticating ADLS Gen2 using a **Service Principal (OAuth2)** inside **Azure Databricks** [Refer Note 6.3 & 7]

[Home](#) > [App registrations](#) >

Register an application ...

* Name

The user-facing display name for this application (this can be changed later).

ecommerce-app-registration-for-databricks ✓

Supported account types

Who can use this application or access this API?

- ☒ Accounts in this organizational directory only (Default Directory only - Single tenant)
- ☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)
- ☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)
- ☐ Personal Microsoft accounts only

[Help me choose...](#)

Go to the certificates and secrets and create a new client secret. [For more go to Note 6.]

Add a client secret ✕

Description

databricks-client-secret

Expires

Recommended: 180 days (6 months) ▼

Then create Access Control IAM for *ecommercedatastorage9>ecom-data*

[Home](#) > [ecommercedatastorage9 | Containers](#) > [ecom-data](#)

ecom-data | Access Control (IAM) ...

Container

◊ ◀ + Add ▼ ⬇ Download role assignments ≡ Edit columns ↻ Refresh | 🗑 Delete | 🗨 Feedback

 Overview

 Diagnose and solve problems

 Access Control (IAM)

> Settings

Check access

Role assignments

Roles

Deny assignments

Classic administrators

My access

View my level of access to this resource.

[View my access](#)

Add role assignment ...

- Role
- Members
- Conditions
- Review + assign

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

 Copilot can help pick a role

- Job function roles
- Privileged administrator roles

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

Storage Blob Data Contributor

Type : All

Category : All

Name ↑↓	Description ↑↓
Storage Blob Data Contributor	Allows for read, write and delete access to Azure Storage blob containers and data

Add role assignment ...

- Role
- Members
- Conditions
- Review + assign

- Selected role
- Storage Blob Data Contributor
- Assign access to
- User, group, or service principal

Managed identity

Members

+ Select members

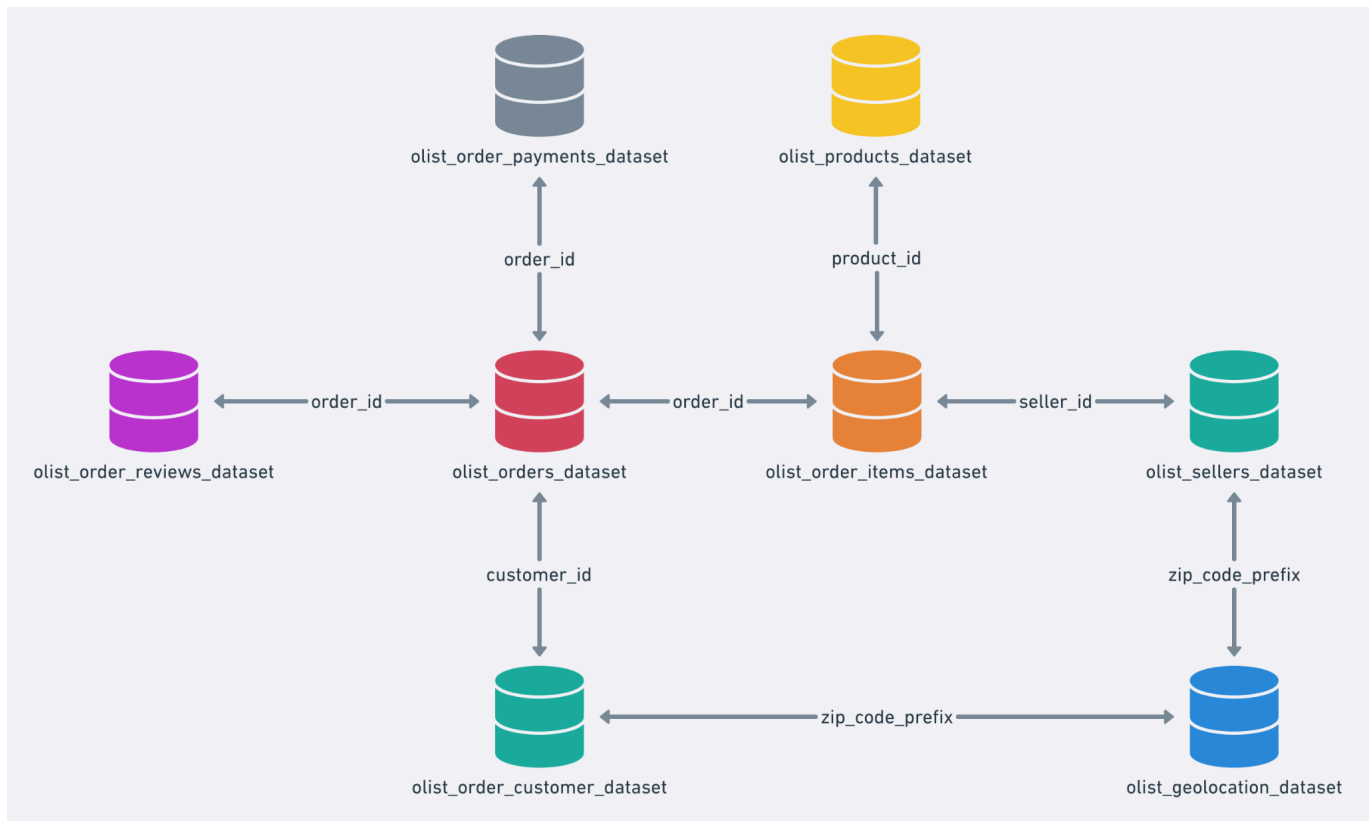
Name	Object ID	Type
ecommerce-app-registration-for-databr...	9abb92b3-cb82-4e36-b07a-16535d207...	App

Description

Optional

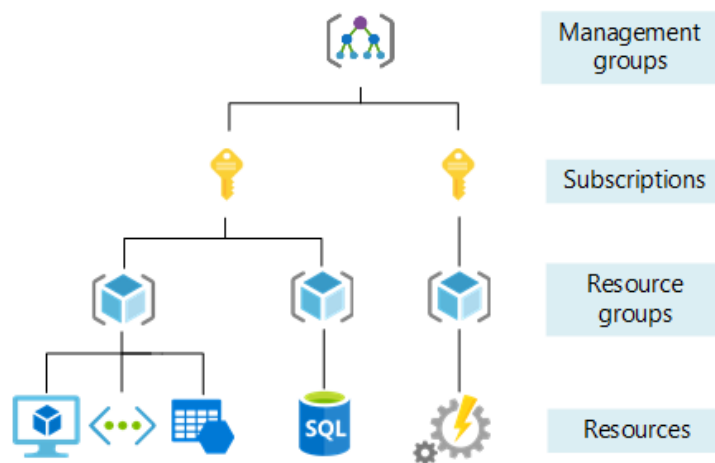
Dataset

1.Data Schema



Extra Notes

1. Azure Account Structure



[Resource organization design area overview](#)

When you start working with **Microsoft Azure**, one of the most important things to understand is how your account and resources are structured. Azure is not just a single bucket where you throw everything in. Instead, it follows a **hierarchical structure** that allows you to organize, manage, and control resources in a secure and cost-effective way.

1.1 Management Groups – Organizing at Scale

At the very top of Azure's structure, above subscriptions, are **Management Groups**.

Management Groups are like **folders for subscriptions**. They don't hold resources directly but instead help you organize **multiple subscriptions** under a common umbrella. This is especially useful for large organizations with many teams, departments, or environments.

For example, a company may create a management group for each department: **HR, Finance, and IT**. Inside each group, they can place subscriptions that belong to that department. Policies and access permissions can then be applied at the management group level, automatically affecting all the subscriptions and resources under it.

In short, **Management Groups help you govern and organize at scale**.

1.2 Subscriptions – The Billing and Access Boundary

A **Subscription** in Azure is where billing and access control start to matter. Every subscription is tied to an **Azure account** and has its own **unique ID, billing account, and quotas**.

Think of a subscription as a **separate folder for projects or environments**. All resources inside a subscription are billed together, and administrators can control access at this level.

Organizations often create different subscriptions for:

- **Development** (to test features)
- **Testing** (to ensure stability)
- **Production** (for customer-facing applications)

This separation makes it easier to manage costs, prevent accidental disruptions, and assign the right access to the right teams.

In other words, **Subscriptions are the dividing lines between budgets, projects, and environments**.

1.3 Resource Groups – Logical Containers for Resources

Within each subscription, we find **Resource Groups**. A resource group is a **logical container** that holds related resources.

The key idea is that resources with the **same lifecycle** should be placed in the same resource group. For example, if you're building a web application, you might put the web server, database, storage account, and virtual network all in one resource group. This way, you can manage them together—if the application is deleted, everything inside that group can be removed at once.

Resource groups also make it easier to apply **permissions, policies, and tags** at a smaller scale than a subscription. This ensures that teams can organize resources without mixing them with unrelated projects.

Thus, **Resource Groups help structure and simplify the management of related resources.**

1.4 Resources – The Building Blocks of Azure

Finally, at the bottom of the hierarchy are the **Resources** themselves. These are the actual services and components you use in Azure, such as:

- Virtual Machines
- Databases
- Web Applications
- Storage Accounts
- Virtual Networks

Resources are always created inside a resource group, which in turn belongs to a subscription. They inherit access permissions and policies from their parent containers.

In short, **Resources are the building blocks of your cloud solutions**—the things you directly work with when running applications in Azure.

1.5 Putting It All Together

To summarize, Azure's account structure follows a clear hierarchy

- **Management Groups** organize multiple subscriptions.
- **Subscriptions** define billing and access boundaries.
- **Resource Groups** provide logical containers for related resources.
- **Resources** are the actual services you use.

Imagine Azure like a company building:

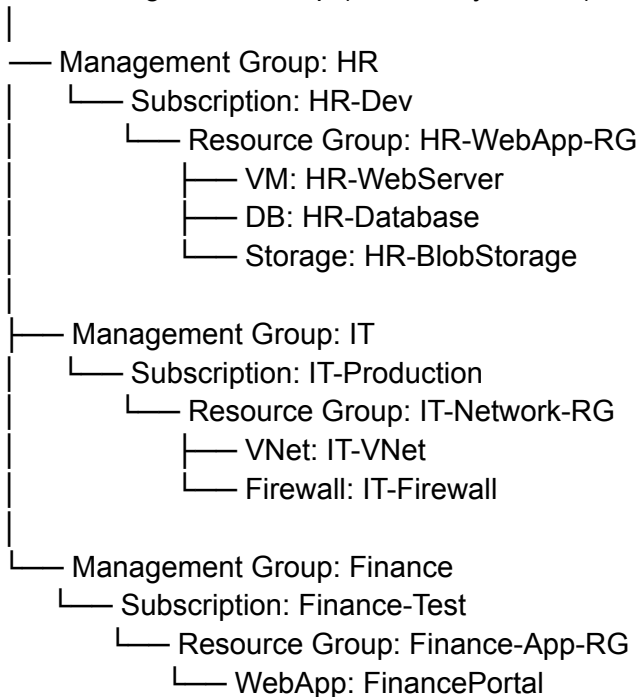
- The **Management Groups** are the departments.
- The **Subscriptions** are the floors of the building.
- The **Resource Groups** are the rooms on each floor.
- The **Resources** are the desks, computers, and equipment inside those rooms.

1.6 Best Practices

1. Use **Management Groups** if your organization has many teams or departments.
2. Keep **different Subscriptions** for development, testing, and production to avoid confusion.
3. Organize **Resource Groups** around applications or lifecycles, not just resource types.
4. Use **tags** to track ownership, cost centers, or environments.
5. Apply **policies and role-based access control (RBAC)** at the management group or subscription level for consistent governance.

1.7 Visual Hierarchy

Root Management Group (created by default)



1.8 Quick Analogy

Think of Azure like a **Company Office Building**:

- **Management Group** → Departments (HR, IT, Finance)
- **Subscription** → Floors of the building (each has separate bills)
- **Resource Group** → Rooms inside a floor (meeting room, server room, workspace)
- **Resources** → Actual items in the room (desks, computers, printers, servers)

1.9 Best Practices for Beginners

1. **Use Management Groups** if you have multiple teams or departments.
2. **Separate Subscriptions** for Dev/Test/Prod environments.
3. **Organize Resource Groups** by application or lifecycle (not by resource type).
4. **Use Tags** (e.g., Project=WebApp, Owner=TeamA) to track costs.
5. **Apply Policies** at management group or subscription level for compliance.

2. Azure Data Factory

2.1 Introduction to Azure Data Factory

- **What is ADF?**
Azure Data Factory (ADF) is Microsoft's cloud-based **data integration and ETL (Extract, Transform, Load) service**.
It helps you **move data, transform it, and load it** into a destination (like a database, data warehouse, or data lake).

Think of ADF as a **factory** where raw materials (data) are brought in, processed, and sent out as finished products (usable insights).

- **Why use ADF?**

- Serverless (no infrastructure management).
- Can handle massive amounts of data.
- Works across on-premises, cloud, and hybrid systems.
- Has a visual drag-and-drop UI for beginners.

- **Real-life analogy:**

Imagine you're managing a factory that makes juice.

- Raw fruits = Raw data (from files, APIs, databases).
- Washing, cutting, blending = Transformations.
- Juice packed into bottles = Final data stored in databases or warehouses.
- Conveyor belts = Pipelines that connect everything.

2.2 Core Concepts of ADF

1. Pipelines

- A pipeline is a **workflow** or a **recipe** that defines how data moves and transforms.
- Example: Copy data from SQL Server → Transform using Data Flow → Store in Azure Data Lake.

2. Activities

- An **activity** is a single step in the pipeline.
- Types of activities
 - **Copy activity:** Move data from one place to another.
 - **Data Flow activity:** Transform data (joins, filters, aggregations).
 - **Notebook activity:** Run Databricks notebooks for advanced processing.

3. Datasets

- A dataset represents **the structure of data** (like a table or file schema).
- Example: A dataset pointing to a CSV file in Azure Blob Storage.

4. Linked Services

- Connection strings that tell ADF **where to find your data**.
- Example: A linked service to an Azure SQL Database or Amazon S3 bucket.

5. Triggers

- Define **when a pipeline runs**.
- Types
 - **Schedule trigger** (every day at 12 AM).
 - **Tumbling window trigger** (every hour).
 - **Event-based trigger** (when a file arrives in storage).

2.3 Building Pipeline

1. **Step 1 – Create a Data Factory**
 - In the Azure portal → Search “Data Factory” → Click *Create*.
2. **Step 2 – Create a Pipeline**
 - Open ADF Studio → Go to “Author” → New Pipeline.
3. **Step 3 – Add Activities**
 - Drag and drop **Copy Activity**.
 - Configure **Source dataset** (e.g., CSV file in Blob Storage).
 - Configure **Sink dataset** (e.g., SQL Database table).
4. **Step 4 – Debug & Publish**
 - Debug (test) your pipeline
 - Publish to save changes.
5. **Step 5 – Trigger Execution**
 - Run pipeline manually or with a trigger.

2.4 Data Transformation in ADF

- **Mapping Data Flows**
 - Visual drag-and-drop interface to transform data.
 - Common transformations:
 - Filter (select only needed rows).
 - Join (combine tables).
 - Aggregate (group and sum).
 - Derived column (calculate new values).
- **Wrangling Data Flows**
 - Similar to **Power Query** in Excel.
 - More user-friendly for non-developers.

2.5 Monitoring & Management

- ADF provides **real-time monitoring** for pipelines.
- You can check
 - Pipeline run status.
 - Error messages.
 - Data movement speed.
- **Alerts & Logs**

- Integrate with Azure Monitor or Log Analytics to get alerts when pipelines fail.

2.6 Real-World Use Cases of ADF

1. Data Migration

- Move on-premises SQL Server data to Azure SQL Database.

2. ETL for Analytics

- Extract data from APIs → Transform with Data Flow → Load into Azure Synapse Analytics.

3. Data Lake Ingestion

- Store raw JSON files into Azure Data Lake, then process them for reporting.

4. Hybrid Data Integration

- Use ADF with **Self-Hosted Integration Runtime** to connect on-premise servers.

2.7 Best Practices

- Organize pipelines logically (naming conventions).
- Use **parameterized pipelines** for reusability.
- Always enable **retry policies** for activities.
- Secure linked services with **Azure Key Vault**.
- Monitor pipelines regularly with alerts.

2.8 Pricing & Cost Optimization

- ADF pricing depends on
 - **Pipeline orchestration** (per activity run).
 - **Data movement** (per data copy).
 - **Data Flow execution** (cluster usage).
- Cost-saving tips:
 - Minimize unnecessary runs.
 - Reuse pipelines with parameters. Use triggers wisely (batch instead of real-time when possible).

2.9 Comparison with Other Tools

Tool	Type	Use Case Example
Azure Data Factory	Cloud ETL/ELT	Enterprise data pipelines
SSIS	On-prem ETL	Legacy SQL Server ETL
Databricks	Big data & ML processing	Advanced analytics
Apache Airflow	Open-source orchestration	Complex workflow scheduling

✓ In Summary

Azure Data Factory = **No-code/Low-code data integration service**.

It connects multiple data sources, transforms data, and loads it into destinations.

It's beginner-friendly but powerful enough for enterprise-scale pipelines.

2.10 Data Factory vs Data Lake vs Data Warehouse

Feature / Aspect	Azure Data Factory (ADF)	Data Lake (Azure Data Lake Storage – ADLS)	Data Warehouse (Azure Synapse Analytics or similar)
Definition	A cloud-based data integration & orchestration tool	A centralized storage repository for raw and processed data	A structured database system optimized for analytics
Primary Purpose	Move, transform, and orchestrate data workflows	Store huge volumes of raw/unstructured/semi-structured data	Store structured, cleaned, and modeled data for reporting & BI
Type of Service	ETL / ELT service	Storage service	Analytics & querying service
Data Types Supported	Structured, semi-structured, unstructured (via connectors)	Structured, semi-structured, unstructured (JSON, CSV, Parquet, images, videos, logs)	Primarily structured (tables, columns, rows)

Data State	Moves/transforms data between sources	Raw/untouched or lightly processed data	Curated, cleaned, aggregated data
Storage Capacity	Doesn't store data; just processes/moves it	Practically unlimited, cost-effective storage	Scalable but more expensive compared to a lake
Processing	Orchestration engine (delegates processing to compute services like Data Flows, Databricks, Synapse)	No processing (just storage)	SQL-based processing (high-performance queries)
Use Cases	- ETL/ELT pipelines- Data migration- Automation workflows	- Store raw IoT logs- Big data archival- Data science exploration	- Business intelligence- Dashboard reporting- Advanced analytics
Integration	Connects with 90+ sources (on-prem & cloud)	Often used as staging area for Data Factory pipelines	Often final destination for pipelines feeding Power BI
Users	Data engineers, integration specialists	Data engineers, data scientists	Analysts, BI developers, decision makers
Cost Model	Pay per activity run & data movement	Pay per TB stored	Pay for compute + storage (DWU in Synapse)
Analogy	Conveyor system in a factory that moves & processes items	Raw materials warehouse storing everything	Finished goods warehouse, organized & ready for sales/insights

✓ In simple terms

- **Data Factory** = The **ETL/ELT engine** (moves and transforms data).
- **Data Lake** = The **storage pool** (keeps raw and historical data).
- **Data Warehouse** = The **organized library** (optimized for fast analytics and business reports).

2.11 Azure Data Factory (ADF) – Authoring Resources

When you work in Azure Data Factory, you "author" different components that together form your **data pipelines**. Each resource has a specific purpose in **data integration, transformation, and movement**.

1. Pipeline

A pipeline is like a workflow container that holds a series of activities (steps).

- Think of it as a recipe for moving and transforming data.
- A pipeline can orchestrate multiple tasks: copy data, transform, load into warehouse, trigger alerts.

Example Use Case

- Ingest data from an on-prem SQL Server → transform it with Data Flow → load into Azure Synapse Analytics.

2. Change Data Capture (CDC)

CDC is a technique that tracks changes (inserts, updates, deletes) in a source system and only moves those changes instead of reloading the entire dataset.

- Saves time and resources.
- Keeps downstream systems in near real-time sync with the source.

Example Use Case

- A retail company updates prices in a POS system.
- Instead of reloading the entire product catalog, CDC pipelines only send the changed prices to the reporting database.

3. Dataset

A dataset defines the structure and location of your data inside ADF.

- Example: a dataset could represent a table in SQL, a CSV file in Blob Storage, or a folder in Data Lake.
- Pipelines and activities use datasets as inputs and outputs.

Example Use Case

- Input dataset = Customer.csv in Blob Storage.
- Output dataset = Customers table in Azure SQL Database.
- ADF pipeline copies data from input → output.

4. Data Flow (Mapping Data Flow)

Data Flows are visual, drag-and-drop transformations inside ADF.

- You don't need to write custom code (though you can if needed).
- Supports filtering, joining, aggregating, pivoting, deduplicating, etc.

Example Use Case

- Cleanse customer data by removing duplicates, standardizing phone numbers, and joining with regional data before loading into Synapse Analytics.

5. Power Query (Data Wrangling)

Power Query inside ADF allows you to prepare and clean data using a familiar Excel/Power BI-style interface.

- Best for business analysts or users who are comfortable with Power BI but not coding.
- Generates transformations automatically in the background.

Example Use Case

- A marketing team uploads campaign data in Excel.
- Using Power Query in ADF, they clean the column names, remove null rows, and enrich with external datasets before sending it to a warehouse.

6. Copy Data Tool

A wizard-driven tool in ADF that helps you quickly copy data from a source to a destination with minimal setup.

- Great for simple ingestion pipelines without much transformation.
- Supports one-time copy or scheduled recurring loads.

Example Use Case

- Copy sales data from Amazon S3 to Azure Data Lake Storage Gen2 for reporting.
- Set it to run daily at midnight.

7. Comparison Table – ADF Authoring Resources

Resource	Purpose	Example Use Case
Pipeline	Workflow container for orchestrating data movement and transformation	Move data from SQL → Data Lake → Synapse
Change Data Capture (CDC)	Track and move only incremental changes	Sync product price updates from POS to reporting DB
Dataset	Defines the structure/location of data (tables, files, folders)	Customer.csv → Customers table in Azure SQL
Data Flow	Visual, code-free transformations (joins, filters, aggregates)	Cleanse and enrich customer data before loading
Power Query	Excel/Power BI–style data wrangling	Marketing team cleans Excel campaign data
Copy Data Tool	Wizard for quick ingestion without complex logic	Copy S3 sales data → ADLS daily

Quick Takeaway

- Use Pipeline to orchestrate workflows.
- Use CDC for incremental updates.
- Use Dataset to define sources and sinks.
- Use Data Flow for data transformations.
- Use Power Query if you prefer a visual/Excel-like experience.
- Use Copy Data Tool for quick and simple ingestion.

8. Azure pipeline building blocks

Component	Description	Options & Configurations	Use Cases	Key Considerations
Copy Data	Moves data from a source to a destination (ETL/ELT foundation).	- Source connectors (SQL, Blob, Data Lake, REST, SAP, etc.)- Sink connectors (SQL DB, Synapse, Cosmos DB, Parquet, etc.)- Performance tuning (parallel copies, staging)- Fault tolerance & retries	Ingesting data from on-prem or cloud sources into a data lake or warehouse.	Check data volume size, choose staged copy for large data, and optimize mapping.
Data Flow	Visual, scalable transformations (similar to SSIS but cloud-native).	- Mapping Data Flows (row-level transformations)- Wrangling Data Flows (Power Query style)- Built-in transformations (join, aggregate, pivot, conditional split)- Debug mode for testing	Complex ETL transformations before loading into storage or warehouse.	Requires integration runtime (IR) compute, costs depend on execution time, debugging spins up Spark clusters.
Notebook / Spark Job Definition	Executes Spark jobs using ADF pipeline triggers.	- Run Jupyter or Databricks notebooks- Parameterize inputs- Linked service to Synapse Spark pool or Databricks- Control dependencies with activity outputs	Data science workflows, advanced analytics, custom machine learning preprocessing.	Best for large-scale computations; ensure Spark environment is optimized for workload.
Data Explorer Activity	Connects and queries Azure Data Explorer (ADX/Kusto).	- Run KQL queries- Export results to Blob or Data Lake- Parameterized queries	High-ingestion analytics pipelines, log data queries, telemetry dashboards.	Efficient for time-series/log data, but requires understanding KQL syntax.

Databricks Activity	Triggers jobs on Azure Databricks cluster.	- Run notebooks- Run JAR/Python scripts- Pass dynamic parameters- Choose existing or new interactive cluster	Big data transformations, ML model training, graph processing.	Costs depend on Databricks cluster runtime; clusters should be auto-terminated after runs.
Data Lake Analytics (U-SQL Activity)	Runs U-SQL scripts in Azure Data Lake Analytics.	- Define U-SQL jobs- Specify ADLA account- Parameterize scripts	Legacy big data jobs, large-scale file transformations.	ADLA is being phased out; Synapse & Spark preferred in modern architectures.
General Activities	Common utilities for orchestration and control.	- Execute Pipeline- Web Activity (API call)- Lookup Activity- Get Metadata- Stored Procedure execution	API calls, pipeline chaining, metadata-driven pipelines.	Lightweight but critical for orchestration; consider retry logic and error handling.
HDInsight Activity	Runs jobs on Hadoop clusters.	- Hive, Pig, MapReduce, Spark, or custom scripts- Parameterized inputs- Supports linked services for storage	Legacy Hadoop workloads, batch data transformations.	Mostly replaced by Synapse/Spark; useful for organizations still using Hadoop clusters.
Iteration & Condition Activities	Control flow elements for looping and conditional execution.	- ForEach (parallel/serial loops)- Until activity (loop until condition met)- If Condition (branching logic)- Switch case	Iterating over datasets, applying dynamic rules, branching pipelines.	Watch out for costs when looping over large datasets; parallel execution can speed things up but consumes compute.
Machine Learning Activity	Integrates with Azure ML services.	- Invoke Azure ML pipeline- Retrain or deploy models- Score datasets- Parameterize ML jobs	Automating ML training pipelines, batch inference, retraining models with new data.	Requires Azure ML workspace setup; costs depend on compute type; monitor drift and retraining frequency.

2.12 Azure Pipeline Components - Copy Data

1. Azure Data Factory Linked Service Options

Option	Explanation	Sub-Options	Sub-Options Explanation & Examples
Name	The unique name of the linked service. Used to identify and reference it in pipelines.	<i>Custom text (must be unique)</i>	Example: "LS_AzureSQL_DB", "LS_Blob_Storage_Prod". Choose meaningful names like LS_<System>_<Purpose>.
Connect via Integration Runtime	Defines the compute environment that executes the connection. Controls where and how the service runs.	- AutoResolveIntegrationRuntime (default) - Self-Hosted Integration Runtime Azure SSIS IR	- <i>AutoResolve</i> : Uses Microsoft-managed compute in the closest region. - <i>Self-Hosted</i> : Runs on-premises or VM for connecting to on-prem systems. - <i>Azure SSIS IR</i> : Runs SSIS packages. Example: use Self-Hosted IR to connect to an on-prem SQL Server.
Base URL	The root web address (endpoint) for API-based or REST-linked services.	<i>Custom URL</i>	Example: https://api.github.com/ for GitHub REST API, or https://login.microsoftonline.com/ for Azure AD authentication.
Server Certificate Validation	Ensures secure SSL/TLS connections when connecting to services.	- Enabled (default) - Disabled	- <i>Enabled</i> : Recommended for security, validates SSL cert from the server. - <i>Disabled</i> : Only for testing/self-signed certs. Example: Use Disabled when connecting to a local dev HTTPS server without valid SSL.
Authentication Type	Defines how ADF proves its identity to the source/destination.	- Anonymous - Basic (username/password) - OAuth2- Service Principal (Azure AD App) - Managed Identity	- <i>Anonymous</i> : Public APIs (no auth). - <i>Basic</i> : Supply username+password. - <i>OAuth2</i> : Common for APIs like Microsoft Graph. - <i>Service Principal</i> : Use App ID & secret/certificate.

			- <i>Managed Identity</i>: Use ADF's system identity (recommended). Example: Connecting to Azure Blob using Managed Identity avoids storing credentials.
Auth Headers	Custom HTTP headers for API connections.	<i>Key-value pairs</i>	Example: <code>Authorization: Bearer <token></code> or <code>x-api-key: 12345</code>. Useful for REST APIs requiring keys.
Annotations	Tags or notes that describe the linked service. Helps with organization.	<i>Free text</i>	Example: <code>"Environment: Production"</code>, <code>"Owner: Data Engineering Team"</code>. Makes governance and troubleshooting easier.
Parameters	Variables that make the linked service reusable across multiple environments.	<i>Key-value pairs, dynamic expressions</i>	Example: Define a parameter <code>dbName</code> so one linked service can connect to multiple databases: <pre>Server=tcp:sqlserver.database.windows.net;Database=@{linkedService().dbName}.</pre>
Advanced Settings	Extra configurations for fine-tuning the linked service.	Connection Timeout- Retry Count- Encryption- Proxy settings	- <i>Timeout</i>: Set how long before connection fails. - <i>Retry</i>: Automatic retries if first connection fails. - <i>Encryption</i>: Force SSL/TLS. - <i>Proxy</i>: Route via proxy. Example: Increase timeout for very large queries from SQL Server.

2. Copy Data Options

General Section

Option	Explanation	Sub-Options	Examples & Notes
Name	Unique identifier for the activity.	<i>Custom text</i>	Example: <code>Copy_CSV_To_SQL</code> . Use meaningful names to describe source→target.
Description	Optional note about the activity.	<i>Free text</i>	Example: <code>"Copies sales data from Blob to Azure SQL DB"</code> . Useful for documentation.

Activity State	Whether the activity runs or not.	- Enabled (default)- Disabled	Example: Disable an activity temporarily during testing.
Timeout	Maximum time before the activity fails.	<i>Duration (e.g., 00:30:00)</i>	Example: " 01:00:00 " (1 hour). Useful for long-running transfers.
Retry	Number of retries if it fails.	<i>Integer (0–n)</i>	Example: Retry = 3 . Helps with transient network issues.
Retry Interval	Time between retries.	<i>Duration (e.g., 00:05:00)</i>	Example: Retry interval = 5 minutes.
Secure Output	Hides sensitive values in pipeline outputs/logs.	- True - False	Example: Enable when copying to destinations that include secrets.
Secure Input	Hides sensitive values in input logs.	- True - False	Example: Enable when API request body has passwords/tokens.

Source section

Option	Explanation	Sub-Options	Examples & Notes
Source Dataset	Defines where data comes from.	Any dataset (SQL, Blob, REST, SAP, etc.)	Example: Source dataset = Blob_CSV_Dataset .
Request Method	For REST/HTTP sources, method to fetch data.	- GET - POST - PUT - DELETE	Example: Use GET for fetching records, POST when sending a query.
Additional Headers	Custom headers for API calls.	<i>Key-value pairs</i>	Example: Authorization: Bearer <token> .
Request Body	Payload for POST/PUT API calls.	<i>JSON/XML/text</i>	Example: { "query": "SELECT * FROM sales" }.
Request Timeout	How long to wait for API before failing.	<i>Duration</i>	Example: 00:02:00 (2 minutes).

Max Concurrent Connections	Limits number of simultaneous connections.	<i>Integer</i>	Example: 5 connections to prevent overloading source system.
Skip Line Count	Skips rows at the start of a text/CSV file.	<i>Integer</i>	Example: Skip 1 line for CSV headers.
Additional Columns	Adds extra metadata columns during copy.	<i>Custom column mappings</i>	Example: Add <code>InsertDate = currentDateTime()</code> .

Sink Section

Option	Explanation	Sub-Options	Examples & Notes
Sink Dataset	Defines where data will be written.	Any dataset (SQL, Blob, Data Lake, Cosmos DB, etc.)	Example: Sink dataset = <code>AzureSQL_SalesDB</code> .
Write Behavior	Defines how sink handles existing data.	- Append- Upsert- Overwrite	Example: Append new rows to table, or Overwrite CSV file.
Batch Size / Write Size	Number of rows or MB per write batch.	<i>Integer</i>	Example: Batch size = 10,000 rows.
Pre-copy Script	SQL statement to run before copy.	<i>SQL code</i>	Example: <code>TRUNCATE TABLE staging_sales</code> .

Mapping Section

Option	Explanation	Sub-Options	Examples & Notes
Column Mapping	Maps source fields to sink fields.	- Auto mapping- Manual mapping	Example: Source column <code>cust_id</code> → Sink column <code>CustomerID</code> .
Schema Drift	Allows handling columns not explicitly mapped.	- Enabled- Disabled	Example: Enable when schema changes often (like logs).

User properties section

Option	Explanation	Sub-Options	Examples & Notes
Custom Tags for Monitoring	Add metadata to activities for tracking/alerting.	Key-value pairs	Example: "Environment: Prod", "Owner: DataEngTeam".

3. Azure SQL Database Options

3.1 Azure SQL Database (PaaS – Platform as a Service)

- A **fully managed SQL Server database in the cloud**
- No need to manage hardware, patching, backups, or updates
- Best for **cloud-native apps, SaaS, and microservices**

Deployment Models

1. **Single Database**
 - Isolated database with its own resources
 - Good for lightweight apps or per-customer databases in SaaS
2. **Elastic Pool**
 - Multiple databases share a pool of resources
 - Cost-efficient for workloads with **variable or unpredictable usage**

3.2 Azure SQL Managed Instance (PaaS with near 100% SQL Server compatibility)

- Managed SQL Server instance in the cloud
- Supports most **on-premises SQL Server features** (SQL Agent, cross-database queries, linked servers)
- Ideal for **migrating existing SQL Server workloads with minimal changes**

Use Cases

- Lift-and-shift migrations
- Legacy applications needing SQL Agent or CLR integration

3.3 SQL Server on Azure Virtual Machines (IaaS – Infrastructure as a Service)

- Full **SQL Server installed on a VM** running in Azure
- You manage OS, SQL Server configuration, backups, and patching
- Maximum flexibility, closest to **on-premises SQL Server experience**

Use Cases

- Applications requiring **full control** over SQL Server and OS
- Workloads needing features not yet supported in PaaS offerings

3.4 Specialized Options within Azure SQL Database

- **Hyperscale (Azure SQL DB tier)**
 - Supports databases up to **100 TB**
 - High scalability for read-heavy or very large workloads
- **Serverless (Azure SQL DB compute tier)**
 - Auto-pauses when not in use
 - Scales compute automatically based on demand
 - Cost-saving for intermittent workloads

3.5 Comparison Table: Azure SQL Options

Option	Service Type	Key Features	Best For	Management Responsibility
Azure SQL Database (Single)	PaaS	Fully managed, single DB	New cloud apps, SaaS, microservices	Microsoft manages everything
Azure SQL Database (Elastic Pool)	PaaS	Multiple DBs share resources	SaaS with many small/variable DBs	Microsoft manages everything
Azure SQL Managed Instance	PaaS	Near full SQL Server compatibility, managed backups/patching	Lift-and-shift migrations, legacy workloads	Microsoft manages infra + SQL; user manages schema & jobs
SQL Server on Azure VM	IaaS	Full SQL Server + OS control	Custom setups, unsupported PaaS features, on-premises-like control	You manage OS, SQL config, patching
Azure SQL Hyperscale	PaaS (Tier)	Up to 100 TB, fast scaling, multiple replicas	Very large, read-heavy workloads	Microsoft
Azure SQL Serverless	PaaS (Tier)	Auto-scaling compute, pauses when idle	Intermittent workloads, dev/test environments	Microsoft

Quick Summary

- Use **Azure SQL Database (Single/Elastic Pool)** for **cloud-native new apps**.
- Use **Managed Instance** if migrating **existing on-prem SQL** with minimal changes.
- Use **SQL Server on VM** if you need **full control**.
- Use **Hyperscale** for **massive workloads**.
- Use **Serverless** for **cost-efficient, variable workloads**.

3.6 Azure SQL Authentication Methods

Azure SQL supports **three main authentication approaches**

1. **Microsoft Entra-only Authentication** (formerly Azure AD-only)
2. **Both SQL + Microsoft Entra Authentication** (hybrid)
3. **SQL Authentication** (traditional username/password)

Comparison Table

Authentication Method	Description	Pros 	Cons 	Best Use Case
Microsoft Entra-only Authentication	Uses Microsoft Entra ID (Azure AD) identities for login (users, groups, managed identities). No SQL logins allowed.	Strong security (SSO, MFA, conditional access)- Centralized identity management- Easier governance (role-based access)	Requires Entra ID setup- Legacy apps may not support it- More complex initial configuration	Enterprises using Microsoft 365 / Entra ID for centralized identity and strong compliance
Both SQL + Microsoft Entra Authentication	Supports both Entra ID users and SQL logins simultaneously.	- Flexibility (supports modern + legacy apps)- Easier migration path- Can gradually move to Entra-only	- Still exposes SQL login (less secure)- Dual management overhead	Organizations migrating from SQL authentication to Entra, hybrid environments
SQL Authentication	Traditional username + password stored inside SQL Database.	- Simple and widely supported- Works with legacy apps/tools- No dependency on Entra ID	- Weak security (password-based)- No MFA, SSO, or conditional access- Harder to govern at scale	Legacy applications, dev/test environments, when Entra ID is not available

Quick Recommendations

- Use **Entra-only** if: security, compliance, MFA, and centralized identity are priorities.
- Use **Both** if: migrating from SQL authentication but still need legacy support.
- Use **SQL Authentication** only if: you have **legacy apps** that don't support Entra or in **temporary/test environments**

3.7 Networking, Security, and Additional Settings

Networking

Azure SQL offers multiple ways to control how clients connect

- Public Endpoint (default)
 - Database accessible over the internet with firewall rules (IP allowlist).
 - Easy but less secure.
- Private Endpoint (Private Link)
 - Connects SQL Database to your VNet (Virtual Network) securely.
 - Prevents exposure to the public internet.
- Service Endpoints
 - Secures traffic between Azure VNet and SQL Database over Azure backbone.
 - Less secure than Private Link, but simpler to set up.

Security

Azure SQL provides built-in enterprise-grade security features

- Authentication
 - Microsoft Entra ID (preferred)
 - SQL Authentication (username/password)
 - Hybrid (both)
- Authorization
 - Role-based access (e.g., db_owner, db_reader, db_datawriter)
 - Entra groups for easier management
- Encryption
 - TDE (Transparent Data Encryption): Encrypts data at rest automatically.
 - Always Encrypted: Protects sensitive columns (e.g., SSNs, credit cards).
 - TLS/SSL: Encrypts data in transit.
- Auditing & Threat Detection
 - Logs DB activities for compliance.
 - Advanced Threat Protection alerts on suspicious activities (SQL injection, brute force).

Additional Settings

When creating an Azure SQL Database, you'll also configure

- Collation
 - Defines sorting and string comparison rules (e.g., case-sensitive or not).
 - Must be chosen carefully upfront.
- Maintenance Window
 - Choose a preferred update window to control when Azure applies patches.
- Backup & Restore
 - Automated backups with point-in-time restore (7–35 days for SQL DB; up to 10 years with Long-Term Retention).
- Geo-Replication
 - Create readable replicas in different regions for disaster recovery or global apps.
- Failover Groups
 - Automatic failover for high availability across regions.
- Database Options
 - Serverless compute, Hyperscale, Elastic Pools.
 - Performance tiers (DTU-based or vCore-based).

Category	Option	Description	Best For
Networking	Public Endpoint	Access via internet with firewall	Simple apps, dev/test
	Private Endpoint (Private Link)	Private IP inside VNet	Secure production workloads
	Service Endpoints	Secure over Azure backbone (not internet)	Mid-level security, easy setup
Security	Authentication	SQL, Entra-only, or both	Flexible depending on app needs
	Encryption	TDE, Always Encrypted, TLS	Compliance, sensitive data
	Threat Protection	Auditing, alerts, anomaly detection	Enterprises needing compliance

Additional Settings	Collation	String comparison rules	Internationalization, case sensitivity
	Backup & Restore	Automated + point-in-time restore	Disaster recovery
	Geo-Replication	Read replicas in other regions	Global applications
	Failover Groups	Auto failover across regions	High availability
	Compute Tiers	DTU, vCore, Serverless, Hyperscale	Workload optimization

Quick Takeaways

- Always use **Private Endpoint** for production (Public only for dev/test).
- Use **Microsoft Entra authentication** for enterprise security.
- Enable **TDE, auditing, and threat detection** for compliance.
- Configure **geo-replication + failover groups** for global and mission-critical apps.

3.8 Connection Establishment

Database Type Determines the Library

Database Type	Default Port	Python Library Example	Notes
MySQL / MariaDB	3306	<code>mysql-connector-python</code> , <code>PyMySQL</code>	Uses MySQL protocol.
SQL Server / Azure SQL DB	1433	<code>pyodbc</code> , <code>pymssql</code>	Uses TDS (Tabular Data Stream) protocol. MySQL libraries cannot speak TDS.

Your server `ecommerce-sql-1.database.windows.net:1433` is **Azure SQL Database**, which is **Microsoft SQL Server**, not MySQL.

`mysql-connector-python` **cannot connect** because it only understands the MySQL protocol on port 3306.

Why **pyodbc** Works for Azure SQL

- **pyodbc** uses **ODBC drivers** that can communicate with SQL Server's **TDS protocol**.
- Supports **encryption**, **Azure authentication**, **timeouts**, and all SQL Server data types.
- Recommended and officially supported by Microsoft for Python → Azure SQL connectivity.

3.9 Send Data From CSV

```
# Import Data
import pandas as pd
order_payments = pd.read_csv('olist_order_payments_dataset.csv')
order_payments.head(5)
```

	order_id	payment_sequential	payment_type	payment_installments	payment_value
0	b81ef226f3fe1789b1e8b2acac839d17	1	credit_card	8	99.33
1	a9810da82917af2d9aefd1278f1dcfa0	1	credit_card	1	24.39
2	25e8ea4e93396b6fa0d3dd708e76c1bd	1	credit_card	1	65.71
3	ba78997921bbcdc1373bb41e913ab953	1	credit_card	8	107.78
4	42fdf880ba16b47b59251dd489d4441a	1	credit_card	2	128.45

```
import pandas as pd

import pyodbc

# Connection details (Azure SQL)
server = "ecommerce-sql-1.database.windows.net,1433"

database = "ecommerce-sql-db-1"

username = "admin123"

password = "ecomdp#@@S"


# CSV file path
csv_file_path = "olist_order_payments_dataset.csv"


# Table name
table_name = "order_payments"


# Connection string
conn_str = f"""
DRIVER={{ODBC Driver 18 for SQL Server}};
```

```
SERVER={server};

DATABASE={database};

UID={username};

PWD={password};

Encrypt=yes;

TrustServerCertificate=no;

Connection Timeout=30;

"""

try:

    # Step 1: Connect to SQL Server

    conn = pyodbc.connect(conn_str)

    cursor = conn.cursor()

    print("Connected to SQL Server successfully!")

    # Step 2: Drop table if exists

    cursor.execute(f"IF OBJECT_ID('{table_name}', 'U') IS NOT NULL DROP TABLE {table_name};")

    print(f"Table '{table_name}' dropped if it existed.")

    # Step 3: Create table

    create_table_query = f"""

    CREATE TABLE {table_name} (

        order_id NVARCHAR(50),

        payment_sequential INT,

        payment_type NVARCHAR(20),

        payment_installments INT,

        payment_value FLOAT

    );

    """

    cursor.execute(create_table_query)

    print(f"Table '{table_name}' created successfully!")

    # Step 4: Load CSV into pandas DataFrame

    df = pd.read_csv(csv_file_path)
```

```

print("CSV data loaded into pandas DataFrame.")

# Step 5: Insert data in batches

batch_size = 500

total_records = len(df)

print(f"Starting data insertion in batches of {batch_size} records...")

for start in range(0, total_records, batch_size):

    end = start + batch_size

    batch = df.iloc[start:end]

    records = [tuple(row) for row in batch.itertuples(index=False, name=None)]

    placeholders = ", ".join("?" * len(batch.columns))

    insert_query = f"INSERT INTO {table_name} VALUES ({placeholders})"

    cursor.fast_executemany = True # Enable fast insert for large batches

    cursor.executemany(insert_query, records)

    conn.commit()

    print(f"Inserted records {start + 1} to {min(end, total_records)} successfully.")

print(f"All {total_records} records inserted into '{table_name}'.")

except Exception as e:

    print("Error:", e)

finally:

    if 'cursor' in locals():

        cursor.close()

    if 'conn' in locals():

        conn.close()

    print("Connection closed")

```

3.10 Better Alternative Ways To send data

A. `cursor.fast_executemany = True` (pyodbc)

- Only works with pyodbc.
- Optimizes `cursor.executemany()` for bulk inserts.
- Great for row-by-row inserts from Python lists or Pandas.

Example

```
import pyodbc

import pandas as pd

conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL
Server};SERVER=...;DATABASE=...;UID=...;PWD=...;Encrypt=yes;")

cursor = conn.cursor()

cursor.fast_executemany = True

df = pd.read_csv("data.csv")

records = [tuple(row) for row in df.itertuples(index=False, name=None)]

placeholders = ", ".join("?" * len(df.columns))

cursor.executemany(f"INSERT INTO table_name VALUES ({placeholders})", records)

conn.commit()
```

Pros: Very fast, direct, fine-grained control.

Cons: Works only with pyodbc, not higher-level ORM.

B. `from sqlalchemy import create_engine` (SQLAlchemy)

- SQLAlchemy provides **ORM-style abstraction** and **connection pooling**.
- Works on top of pyodbc (or other DB drivers) to simplify connections and table operations.
- Can insert **Pandas DataFrames directly** with `df.to_sql()`.

Example

```
from sqlalchemy import create_engine

import pandas as pd

import urllib

server = 'ecommerce-sql-1.database.windows.net'

database = 'ecommerce-sql-db-1'

username = 'admin123'

password = 'ecomdp#@@S'

params = urllib.parse.quote_plus(
    f"DRIVER={{ODBC Driver 18 for SQL Server}};SERVER={server};DATABASE={database};UID={username};PWD={password};Encrypt=yes;TrustServerCertificate=no;"
)

engine = create_engine(f"mssql+pyodbc:///odbc_connect={params}")

df = pd.read_csv("data.csv")

df.to_sql("table_name", engine, if_exists="replace", index=False, chunksize=500)
```

Pros: Very clean, less code, automatic batching (`chunksize`), handles schema automatically.

Cons: Slightly less control than raw pyodbc; may be slower for extremely large datasets unless tuned.

Recommendation

Scenario	Use
You want maximum speed with large batches and full control	<code>pyodbc + fast_executemany=True</code>
You want clean Pandas integration and simpler code	<code>SQLAlchemy + df.to_sql(chunksize=...)</code>

- Can combine both: SQLAlchemy can use a **pyodbc connection with `fast_executemany=True`** for super-fast `to_sql` inserts.

4. Azure MongoDB

4.1 Azure Cosmos DB (MongoDB API)

- Type: Fully-managed NoSQL database with MongoDB API compatibility.
- Protocol: Supports MongoDB wire protocol (versions ~3.6–6.0 depending on API).
- Key Features
 - Global distribution across multiple Azure regions.
 - Fully managed, auto-scaling throughput.
 - Multi-master replication.
 - Enterprise-grade SLAs.
 - Integrated backup, monitoring, and security.
- **Use Cases**
 - Cloud-native apps requiring global distribution.
 - Apps that want MongoDB API without managing servers.
- **Pros**
 - Serverless or provisioned throughput.
 - Fully managed, high availability.
- **Cons**
 - Some MongoDB features not fully supported (e.g., certain transactions, aggregation pipeline stages).
 - Slightly different pricing than self-managed MongoDB.

4.2 MongoDB Atlas on Azure

- **Type:** Managed MongoDB service by MongoDB Inc., deployable on Azure.
- **Key Features**
 - Full MongoDB feature support.
 - Global clusters and multi-region replication.
 - Automated backups and monitoring.
 - Integrated security (VNet peering, private endpoints)
- **Use Cases**
 - Need full MongoDB feature set.
 - Enterprises wanting SLA-backed MongoDB service.
- **Pros**
 - Supports all MongoDB features.
 - Easy scaling, monitoring, and maintenance.
- **Cons**
 - Managed by MongoDB Inc. (separate billing).
 - Slightly higher cost than Cosmos DB in some regions.

4.3 Self-Managed MongoDB on Azure VMs

- **Type:** Install and manage MongoDB yourself on **Azure Virtual Machines**.
- **Key Features**
 - Full control over MongoDB version, configuration, and storage.
 - Can deploy sharded clusters, replica sets, or single nodes.
- **Use Cases**
 - Custom setups not supported by managed services.
 - Organizations with strict regulatory or compliance requirements.
- **Pros**
 - Full flexibility.
 - Can run any MongoDB version or extensions.
- **Cons**
 - You manage scaling, backups, monitoring, and upgrades.
 - Higher operational overhead.

4.4 Comparison Table

Feature	Cosmos DB (Mongo API)	MongoDB Atlas on Azure	Self-Managed MongoDB on VMs
Fully managed	✓	✓	✗
MongoDB feature support	Partial	Full	Full
Multi-region/global distribution	✓	✓	✗ (manual)
Auto-scaling	✓	✓	✗
Backups & monitoring	✓	✓	✗ (manual)
Private endpoint support	✓	✓	✓ (manual)
Cost	Moderate to high	Moderate to high	VM cost + ops
Control over Mongo version	Limited	Full	Full

4.5 Authentication & Networking in Azure

- Authentication
 - Cosmos DB: MongoDB API authentication (SCRAM-SHA-1/256).
 - Atlas: Standard MongoDB auth.
 - Self-managed: User-defined auth and roles.
- Networking
 - Cosmos DB & Atlas: Private endpoints, VNet peering, firewall rules.
 - Self-managed: VMs inside VNet, NSG, firewall, SSL.

4.6 When to Choose Which

- Cosmos DB Mongo API → Cloud-native, low maintenance, globally distributed apps.
- MongoDB Atlas on Azure → Full MongoDB features + managed service.
- Self-managed on Azure VMs → Complete control, custom deployments, or specific versions.

5. Azure Storage

5.1 Azure Storage options

Storage Type	Description	Sub-Types / Options	Use Cases / Notes
Blob Storage	Object storage for unstructured data (text, images, videos, logs).	- Block Blob: Stores large files in blocks (most common). - Append Blob: Optimized for append operations (logging). - Page Blob: Random read/write access, used for disks (VHD).	Store documents, images, videos, backups, logs. Example: Store CSV files from pipelines.
File Storage	Managed file shares accessible via SMB/NFS protocol.	- Standard File Share: General-purpose file share. - Premium File Share: High-performance SSD-backed.	Lift-and-shift legacy applications that require shared network drives. Example: Shared directory for application config files.
Queue Storage	Message queue for decoupling components.	- Single queue type, messages up to 64 KB each.	Communication between services; decoupled processing pipelines. Example: Trigger ADF pipelines based on queued events.
Table Storage	NoSQL key-value storage for semi-structured data.	- PartitionKey / RowKey structure.	Store telemetry, logs, or metadata. Example: Storing IoT device data.
Disk Storage	Managed disks for Azure VMs.	- Standard HDD: Low-cost. - Standard SSD: Better performance. - Premium SSD: High IOPS. - Ultra Disk: Extreme performance, low latency.	Attach to VMs for OS/data drives. Example: Premium SSD for database VMs.
Azure Data Lake Storage Gen2 (ADLS)	Optimized storage for big data analytics.	- Hierarchical namespace (HNS) enabled for directories - Supports POSIX-like ACLs.	Big data processing with Spark, Synapse, HDInsight. Example: Store raw, curated, and enriched data in a lake.
Managed Disks	Simplified management of VM disks.	- OS Disk - Data Disk - Temporary Disk	Automatically handles replication and high availability. Example: Attach to VM for persistent storage.

Premium Storage	High-performance SSD-backed storage for IOPS-sensitive workloads.	- Available for Blob, File, Disk Storage.	High-performance databases, VM workloads, transactional apps.
Azure Archive Storage	Long-term, low-cost cold storage.	- Archive Tier (lowest cost) - Cool Tier (moderate cost) - Hot Tier (frequent access)	Storing backups, compliance data, rarely accessed files. Example: Yearly compliance logs.
Azure NetApp Files	Enterprise-grade NFS/SMB storage for high-performance workloads.	- NFS v3/v4.1 - SMB 3.1.1	SAP, Oracle, HPC workloads requiring fast shared storage.
Premium Blob Storage	SSD-backed block blob storage for low-latency workloads.	- Hot, Cool tiers available	Low-latency transactional workloads for media or analytics.

Notes

1. Choose based on access patterns

- Hot: Frequently accessed.
- Cool: Infrequent access (30+ days).
- Archive: Rarely accessed (180+ days).

2. Big Data: Use **ADLS Gen2** for hierarchical organization and analytics workloads.

3. VMs & Databases: Use **Managed Disks** or **Premium SSD** for OS and high-performance apps.

4. Decoupled Messaging: Use **Queue Storage** to trigger pipelines, events, or background jobs.

6. Azure App Registrations (Azure AD)

App Registration in Azure Active Directory (Azure AD) is how you register an application so that it can authenticate (prove its identity) and access resources (like Microsoft Graph, APIs, or your own backend services).

Think of it like giving your app an identity card inside Azure AD.

When you create an App Registration in Azure Active Directory, you're essentially telling Azure AD, "This app needs an identity." Behind the scenes, Azure AD generates a client ID (like a username) and associates it with your tenant. When a user or application tries to sign in, the app redirects them to Azure AD's authorization endpoint, where authentication happens. If successful, Azure AD issues a token (JWT) that contains information about the user or the app itself. This token is then presented to the resource (like Microsoft Graph, an API, or your own backend) as proof of identity and permissions. The app's secrets or certificates serve as credentials to prove that the request really came from your registered app, while permissions/scopes define what the app is allowed to access. In short, App Registration makes Azure AD the central authority that validates identities and securely hands out tokens for trusted apps to access resources.

6.1 Key Concepts

Concept	Explanation	Example
Application (client) ID	A unique identifier assigned to your app after registration.	"12345678-abcd-90ef-1234-56789a bcdef0"
Directory (tenant) ID	Unique ID of your Azure AD tenant.	"abcd1234-5678-ef90-1234-56789a bcdef0"
Client Secret / Certificate	Credentials your app uses to authenticate. Works like a password (secret) or a certificate.	Store securely in Key Vault .
Redirect URI	Where Azure AD sends the token after authentication.	https://myapp.com/auth/callback
API Permissions	Defines what your app can do (read mail, access Graph API, call another API).	Permission: User.Read (read user profile).
Managed Identity vs App Registration	- <i>Managed Identity</i> : For Azure resources (auto-managed).- <i>App Registration</i> : For any app (custom/manual).	Use Managed Identity for VM → Blob. Use App Registration for custom API.

6.2 Steps to Create an App Registration

6.2.1. Navigate to App Registrations

- Go to **Azure Portal** → **Azure Active Directory** → **App registrations** → **New registration**.

6.2.2. Fill in the Basics

Option	Explanation	Sub-Options	Examples
Name	Friendly name to identify your app.	Any string, unique only in your tenant.	MyWebApp, HRPortalAPI
Supported Account Types	Who can sign in to your app.	- Single tenant → Only users from your Azure AD tenant. - Multitenant → Users from any Azure AD tenant. - Personal Microsoft accounts → Outlook/Hotmail/Xbox accounts. - Both org + personal.	- Single tenant: Internal HR app. - Multitenant: SaaS app serving multiple companies.
Redirect URI (optional)	The endpoint where tokens are sent after login. Needed for web/mobile apps.	- Web → https://myapp.com/auth/callback - SPA → https://localhost:3000/redirect - Mobile/desktop → msal{appId}://auth	- Web app with sign-in page. - React SPA redirect.

6.2.3. After Registration → Overview Page

Once created, you'll see

Item	Explanation	Example
Application (client) ID	Unique ID for your app.	12345abc-6789-def0-1112-131415abcdef
Directory (tenant) ID	Your tenant's ID.	abcd1234-5678-90ef-ghij-1234567890kl
Object ID	Unique ID for this app object in AD.	9f8g7h6i-5j4k-3l2m-n1o0-p987654321

6.2.4. Add Certificates & Secrets

Option	Explanation	Example
Client Secret	Password-like string apps use to authenticate.	value: "abcd12345-..." (expires in 6 months, 1 year, 2 years).
Certificates	Upload a public key certificate. More secure than secrets.	Upload .cer or .pem file.

6.2.5. Configure Authentication

Option	Explanation	Example
Redirect URIs	Add/update redirect endpoints.	https://localhost:5000/signin-oidc
Implicit Grant	(Legacy) allow tokens via URL. Usually disabled now.	Only for old SPAs.
Default client type	Mark app as Public (native) or Confidential (web/API).	Mobile app = Public; Web API = Confidential.

Logout URL	Where user is redirected after logout.	<code>https://myapp.com/logout</code>
Front-channel / Back-channel logout	Handle user sign-out events.	Needed for multi-tab logout sync.

6.2.6. Configure API Permissions

Option	Explanation	Example
Microsoft APIs	Access Microsoft Graph, Office 365, etc.	<code>User.Read</code> , <code>Mail.Read</code>
APIs my org uses	Access APIs registered in your tenant.	<code>HR-LeaveAPI.Read</code>
Delegated permissions	App acts on behalf of user.	A user logs in, app reads their emails.
Application permissions	App acts as itself (no user).	Background service reads org users list.

6.2.7. Expose an API (optional, if your app is an API)

Option	Explanation	Example
Application ID URI	Unique identifier for your API.	<code>api://myapp12345</code>
Scopes	Define permissions other apps can request.	<code>api://myapp12345/.default</code> or <code>access_as_user</code>
Roles	Assignable roles for consuming apps.	<code>Admin</code> , <code>Reader</code>

6.2.8. Configure Branding & Properties

Option	Explanation	Example
Logo	App's logo in consent screen.	PNG/JPG upload.
Publisher Domain	Domain shown on consent screen.	contoso.com
Home Page URL	User's landing page after sign-in.	https://myapp.com/
Terms of Service / Privacy Policy URLs	Legal links for users.	https://myapp.com/privacy

6.2.9. Assign Owners

- Add admins who can manage this registration.

6.2.10. Final Setup

- Copy **Client ID, Tenant ID, and Secret**.
- Use them in your app's authentication code (MSAL, OAuth2, OpenID Connect).

Summary

Creating an App Registration is like giving your app a passport in Azure AD. You configure who can log in, where tokens go, what APIs it can access, and how it proves its identity (secret/cert).

6.3 Authentication Flows Supported

Flow	When to Use	Example
Authorization Code Flow	Interactive sign-in (web apps, SPAs).	User logs into a website.
Client Credentials Flow	App-to-app (no user).	Backend service calling Graph API.
Device Code Flow	Devices without browsers.	IoT device authenticating via code.
Implicit Flow (legacy)	For SPAs, but replaced by Auth Code + PKCE.	Old Angular/React apps.

7. Authentication Methods in Azure

Method	Explanation	Workflow	Strengths	Weaknesses	Example Use Cases
Password-based (Username + Password)	Users sign in with Azure AD credentials.	User enters username & password → Azure AD verifies → access granted.	Simple, widely understood, easy setup.	Weak security (password reuse, phishing), not recommended alone.	Legacy apps, temporary access for low-risk apps.
Passwordless Authentication	Removes need for passwords using alternative credentials.	User authenticates via device (Authenticator app, Windows Hello, FIDO2 key) → Azure AD verifies → token issued.	Strong security, reduces phishing risk, better UX.	Requires modern devices and setup effort.	Enterprise workforce apps, high-security environments.
Multi-Factor Authentication (MFA)	Requires at least 2 methods (something you know + have + are).	User enters password → secondary check (SMS, phone call, Authenticator app) → access granted.	Much stronger than password-only, mitigates phishing.	Extra step for users, SMS/calls vulnerable to SIM-swap.	Banking portals, HR systems, admin accounts.
Windows Hello for Business	Biometric or PIN tied to device TPM.	User signs in with fingerprint, face, or PIN → local device verifies with key → Azure AD trust.	Fast, secure (keys never leave device).	Requires compatible hardware (TPM, camera, etc.).	Corporate laptops, hybrid-joined Windows devices.
Certificate-based Authentication (CBA)	Uses client certificates to authenticate.	Device presents certificate → Azure AD validates against trusted CA → token issued.	Strong identity assurance, good for devices/services.	Complex setup, certificate lifecycle management.	VPNs, legacy apps, highly regulated industries.
Federated Authentication (ADFS, SAML, WS-Fed)	Azure AD trusts another Identity Provider (IdP).	User redirected to IdP (e.g., ADFS) → IdP authenticates → token passed to Azure AD.	Reuse existing IdPs, centralizes identity.	Complexity, dependency on IdP uptime.	Enterprises migrating from on-prem AD, hybrid setups.

OAuth 2.0 / OpenID Connect (OIDC)	Token-based auth for apps & APIs.	App redirects user → Azure AD authenticates → issues tokens (ID, Access, Refresh).	Modern standard, secure, works across devices.	Requires app integration work.	Web & mobile apps using Azure AD for login.
SAML-based Authentication	XML-based standard for SSO.	App redirects to Azure AD → AD issues SAML token → app validates → user signed in.	Widely supported in enterprise apps.	Verbose, less flexible than OAuth/OIDC.	Legacy SaaS apps (Workday, ServiceNow, Salesforce).
Managed Identity (for Azure Resources)	Azure automatically manages identity for services.	Service (VM, Function, Logic App) requests token → Azure AD provides → no secrets needed.	No secrets, high security, easy integration with Azure services.	Limited to Azure-hosted services.	Key Vault access from Function App, SQL Database access from VM.
FIDO2 Security Keys	Hardware-based passwordless authentication.	User inserts key → taps/biometric → Azure AD validates cryptographic challenge.	Very strong, phishing-resistant, portable.	Requires FIDO2-compatible keys/devices.	High-security roles (admins, developers, executives).
Device-based Authentication (Hybrid/Azure AD Join)	Auth trusts device identity as part of login.	Device registered in Azure AD → authentication trusts device compliance.	Seamless sign-in, conditional access policies.	Requires device management (Intune/AD join).	Corporate devices, MDM-enforced security policies.

Key Takeaways

- For **users** → MFA + Passwordless (Authenticator, FIDO2, Windows Hello) are best.
- For **apps** → OAuth2/OIDC + Managed Identity simplify secure access.
- For **legacy apps** → Federation (ADFS, SAML) bridges old with new.
- For **devices/services** → Certificates, Managed Identity, or Device-based auth are standard.

8. Synapse

Configurations

Create Synapse workspace ...

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all of your resources.

Subscription * ⓘ	Azure subscription 1 ✖ The selected subscription is disabled.
Resource group * ⓘ	ecommerce-project Create new
Managed resource group ⓘ	ecommerce-synapse-rg ✓

Workspace details

Name your workspace, select a location, and choose a primary Data Lake Storage Gen2 file system to serve as the default location for logs and job output.

Workspace name *	ecommerce-synapse-workspace ✓
Region *	(Asia Pacific) Southeast Asia ✓
Select Data Lake Storage Gen2 * ⓘ	☒ From subscription ☐ Manually via URL
Account name * ⓘ	(New) synapsestorageecom Create new
File system name *	(New) synapsefilesystem Create new
☒ Assign myself the Storage Blob Data Contributor role on the Data Lake Storage Gen2 account to interactively query it in the workspace.	

Configure security options for your workspace.

Authentication

Choose the authentication method for access to workspace resources such as SQL pools. The authentication method can be changed later on. [Learn more](#)

Authentication method ⓘ

☒ Use both local and Microsoft Entra ID authentication

☐ Use only Microsoft Entra ID authentication

SQL Server admin login * ⓘ

sqladminuser

SQL Password ⓘ

.....✓

Confirm password

.....✓

System assigned managed identity permission

Select to grant the workspace network access to the Data Lake Storage Gen2 account using the workspace system identity. [Learn more](#)

☐ Allow network access to Data Lake Storage Gen2 account. ⓘ

ⓘ

The selected Data Lake Storage Gen2 account does not restrict network access using any network access rules, or you selected a storage account manually via URL under Basics tab. [Learn more](#)

Workspace encryption

Synapse Configs

Sql Server admin Login	sqladminuser
Password	Qwe\$rtyu

8.1. What is Azure Synapse?

Azure Synapse Analytics (formerly known as Azure SQL Data Warehouse) is a cloud-based analytics service provided by Microsoft Azure. It helps organizations store, process, and analyze large amounts of data quickly and efficiently.

Think of it as a data hub where you can bring together data from different sources (databases, files, APIs, IoT devices, etc.), clean and transform it, and then run powerful queries or create reports.

8.2. Key Features

1. Data Integration

- Connects to many data sources (Azure Data Lake, Cosmos DB, SQL Server, APIs, etc.).
- Works closely with **Azure Data Factory** for data pipelines.

2. Data Storage

- Stores structured (tables, rows, columns) and semi-structured data (JSON, Parquet, CSV).
- Uses **Data Lake + SQL-based storage**.

3. Data Processing

- Two main ways to process data:
 - **Serverless SQL Pool** → Pay-per-query, good for ad-hoc analysis.
 - **Dedicated SQL Pool** → Reserved resources, good for predictable workloads.

4. Data Analytics

- Run **SQL queries** to analyze big data.
- Integrates with **Power BI** for dashboards and visualizations.

5. Machine Learning & AI

- Works with Azure Machine Learning, Spark, and Synapse pipelines for predictive analytics.

6. Security

- Built-in **encryption, role-based access control, firewalls, private endpoints**.
- Ensures compliance with data protection standards.

8.3. Core Components of Synapse

• Synapse Studio

A web-based tool where you build and manage your pipelines, queries, and dashboards.

• Pipelines

ETL/ELT workflows to **Extract, Transform, Load** data from various sources.

• SQL Pools

- **Dedicated SQL Pool** → Scalable data warehouse.
- **Serverless SQL Pool** → Query data on demand.

- **Apache Spark Pools**
For advanced data processing, machine learning, and handling unstructured data.
- **Integration with Power BI**
Directly connect to Synapse for business reports and insights.

8.4. How Azure Synapse Works

1. **Data Ingestion** → Bring data from multiple sources (databases, APIs, files).
2. **Data Storage** → Store raw or processed data in Azure Data Lake or SQL pools.
3. **Data Transformation** → Clean, shape, and prepare data using Synapse pipelines or Spark.
4. **Data Querying** → Use SQL queries (serverless or dedicated) for analysis.
5. **Data Visualization** → Connect with Power BI for dashboards and reporting.
6. **Advanced Analytics** → Apply ML/AI models for predictions and deeper insights.

8.5. Advantages of Azure Synapse

- ✓ **Scalable** – Handle terabytes to petabytes of data.
- ✓ **Pay-as-you-go** – Pay only for what you use (serverless) or reserve resources (dedicated).
- ✓ **Unified Platform** – Combines data integration, storage, processing, and analytics in one place.
- ✓ **Fast Performance** – Optimized for big data queries.
- ✓ **Seamless Integration** – Works with other Azure services (Data Lake, Cosmos DB, Power BI, ML).
- ✓ **Strong Security** – Enterprise-grade data protection.

8.6 How Azure Synapse Works in the Backend

8.6.1. Core Architecture

At its core, Azure Synapse is built on top of a **Massively Parallel Processing (MPP)** architecture

- **Control Node**
 - Acts as the "brain" of Synapse.
 - Handles query parsing, optimization, and orchestration.
 - Breaks down SQL queries into smaller tasks and distributes them to compute nodes.
- **Compute Nodes**
 - Worker nodes that actually execute queries.
 - Store and process distributed chunks of data.
 - Use **Data Movement Service (DMS)** to shuffle data between nodes when required.
- **Distributed Data Storage**
 - Data is physically stored in **Azure Data Lake Storage (ADLS Gen2)** in columnar format (based on **PolyBase + Parquet/ORC**).
 - Dedicated SQL Pools use **distributed tables** with different distribution strategies:
 - **Hash-distributed** → spreads rows based on a column value.
 - **Round-robin distributed** → spreads rows evenly.
 - **Replicated** → keeps a full copy on every compute node.

8.6.2. Query Execution Flow

When a user runs a query in Synapse

1. **Query Submission**
 - Query comes from Synapse Studio, SQL client, Power BI, or API.
 - The **Control Node** parses and validates SQL.
2. **Query Optimization**
 - The Control Node creates an **execution plan**.
 - Chooses best join strategy (broadcast, shuffle, etc.) and indexes.
3. **Task Distribution**
 - Execution plan is split into steps.
 - Tasks are sent to **Compute Nodes** via DMS.
4. **Parallel Processing**
 - Each compute node processes its chunk of data.
 - Uses **columnar storage + compression** for efficiency.
5. **Data Movement**
 - If needed, rows are shuffled between nodes (e.g., for joins on distributed tables).
 - DMS handles this with minimal latency.
6. **Result Aggregation**
 - Partial results are sent back to Control Node.

- Control Node merges results and sends back to the client.

8.6.3. Two Query Models

- **Dedicated SQL Pools (Provisioned)**
 - Pre-allocated compute nodes (DWUs = Data Warehouse Units).
 - Strong performance, predictable workloads.
 - Best for large-scale, frequent queries.
- **Serverless SQL Pools (On-Demand)**
 - No pre-provisioned compute.
 - Query directly against files in ADLS using distributed SQL.
 - Each query spins up temporary compute resources, then releases them.
 - Best for exploratory, ad-hoc queries.

8.6.4. Integration Layer

- **PolyBase Engine**
 - Allows Synapse to read data directly from external sources (Data Lake, Blob, Cosmos DB, external SQL DBs).
 - Uses data virtualization without full ingestion.
- **Pipelines (with ADF engine)**
 - Uses the same orchestration engine as **Azure Data Factory**.
 - Handles ETL/ELT operations, scheduling, and dependency tracking.
- **Apache Spark Pools**
 - Run distributed Spark jobs on top of Synapse-managed clusters.
 - Useful for big data transformation, ML training, or processing unstructured data.

8.6.5. Performance Optimizations

- **Columnstore Indexes** – Data stored in compressed columnar format for fast analytical queries.
- **Result Set Caching** – Recently executed query results can be cached.
- **Materialized Views** – Pre-computed summaries for frequent queries.
- **Workload Management** – Assigns priorities (via workload groups) to balance resources between users.
- **Partitioning** – Large tables are partitioned across nodes for parallelism.

8.6.6. Security & Governance

- **Authentication:** Azure AD + Role-based access.
- **Data Security:** Always-on encryption, firewall rules, private endpoints.
- **Data Lineage:** Synapse integrates with **Purview** for tracking data origins.
- **Monitoring:** Integrated with **Azure Monitor + Log Analytics** for performance tracking.

8.6.7. High-Level Backend Workflow

User Query → Control Node (SQL Frontend)

- Query Parser & Optimizer
- Task Distribution via DMS
- Compute Nodes (MPP parallel execution)
- Data Movement (if needed)
- Aggregation at Control Node
- Final Results to User (Power BI, SQL client, API)

8.7. Azure Data Factory vs Azure Synapse Analytics

Aspect	Azure Data Factory (ADF)	Azure Synapse Analytics
Definition	A data integration & orchestration tool for moving, transforming, and loading data (ETL/ELT).	A data analytics & warehousing platform for storing, querying, and analyzing data.
Main Purpose	To ingest and transform data from multiple sources into a centralized location.	To store and analyze big data using SQL, Spark, and BI integrations.
Core Function	Orchestration of pipelines (Extract, Transform, Load).	Querying and analyzing large-scale structured & semi-structured data.
Data Handling	Moves data from source to target (e.g., SQL Server → Data Lake → Synapse).	Stores data in Dedicated SQL Pools or queries directly with Serverless SQL Pools .
Processing Engine	Uses Mapping Data Flows (Spark-based) or external compute (Databricks, HDInsight).	Uses MPP (Massively Parallel Processing) with Control Node + Compute Nodes.
Storage	Does not store data itself; only orchestrates movement and transformation.	Provides data storage in distributed tables (columnar format in ADLS).
Integration	Connects to 90+ sources (databases, APIs, SaaS, on-prem, cloud).	Deep integration with Power BI, ML, Data Lake, Cosmos DB .
Best For	Building data pipelines , ETL/ELT workflows, data movement.	Big data analytics , BI dashboards, ML-driven insights.
Pricing Model	Pay-per-activity (number of pipeline runs, data flow usage).	Pay-per-query (serverless) or provisioned compute (DWUs for dedicated pools).
Example Use Case	Copy daily sales data from ERP → Clean in Data Flow → Load into Synapse.	Run a SQL query to find top 10 products by revenue → Build Power BI dashboard.

In short

- **ADF = Data movement + transformation tool (ETL/ELT)**
- **Synapse = Data storage + analytics engine (Data Warehouse).**
They often work **together** → ADF loads data into Synapse, and Synapse analyzes it.

8.8 Comparison: Dedicated SQL Pool vs Serverless SQL Pool for Synapse

Aspect	Dedicated SQL Pool	Serverless SQL Pool
Definition	A provisioned data warehouse with fixed compute and storage resources (DWUs).	An on-demand query service that lets you query data directly in Data Lake without provisioning.
Compute Model	Provisioned → You pre-allocate compute (DWUs = Data Warehouse Units).	On-Demand → Compute is allocated at query runtime and released after execution.
Data Storage	Stores data in distributed tables (hash, round-robin, replicated) inside Synapse.	Does not store data ; queries files directly in Azure Data Lake (Parquet, CSV, JSON) .
Performance	Very high performance for large, frequent, complex queries (optimized MPP).	Performance depends on query size; great for exploratory/ad-hoc queries .
Cost Model	Pay for reserved compute (per hour) + storage. Even idle warehouses cost unless paused.	Pay per query (per TB of data scanned). No cost when idle.
Best Suited For	Enterprise-scale workloads, predictable analytics, high concurrency.	Lightweight, exploratory analytics, quick insights, low-frequency queries.
Scalability	Manually scale DWUs up/down or pause when not in use.	Automatically scales based on query size (serverless).
Integration	Works seamlessly with Power BI, ML, Pipelines for BI dashboards and advanced analytics.	Ideal for quick analysis of raw data in Data Lake without loading into a warehouse.
Example Use Case	Storing 10 years of sales data → Running daily reports for business units.	Running a quick SQL query over CSV files in ADLS to check yesterday's sales.

In short:

- **Dedicated SQL Pool = Data Warehouse** (provisioned, powerful, best for ongoing BI).
- **Serverless SQL Pool = Query-on-demand** (cheap, flexible, best for ad-hoc exploration).

8.9 CETAS in Azure Synapse

8.9.1. What is CETAS?

CETAS = **CREATE EXTERNAL TABLE AS SELECT**

It is a **one-step T-SQL command** that

1. Executes a **SELECT query** inside your **Dedicated SQL Pool**.
2. Writes the result set **in parallel** to **Azure Data Lake Storage (ADLS) Gen2 or Blob Storage** in a chosen file format.
3. Creates a new **external table** pointing to that folder of files.

Unlike **INSERT INTO EXTERNAL TABLE**, CETAS is optimized for **export** and writes data more efficiently.

8.9.2. Supported File Formats

CETAS supports

- **PARQUET** (best for analytics, columnar, compressed)
- **ORC** (similar to Parquet, columnar format)
- **DELIMITEDTEXT (CSV/TSV)**

8.9.3. Core Syntax

```
CREATE EXTERNAL TABLE [schema_name].[table_name]
WITH (
    LOCATION = 'folderPath/',
    DATA_SOURCE = [ExternalDataSourceName],
    FILE_FORMAT = [ExternalFileFormatName],
    REJECT_TYPE = VALUE | PERCENT,
    REJECT_VALUE = 0 -- tolerance for bad records
)
AS
SELECT column1, column2, aggregate_function(column3)
FROM dbo.MyTable
WHERE column4 = 'SomeCondition';
```

8.9.4. Setup Requirements

Before running CETAS, you need

External Data Source → tells Synapse where the files will be written.

```
CREATE EXTERNAL DATA SOURCE MyDataLake
WITH (
    TYPE = HADOOP,
    LOCATION = 'abfss://mycontainer@mydatalake.dfs.core.windows.net/'
);
```

External File Format → defines output format (Parquet, ORC, CSV).

```
CREATE EXTERNAL FILE FORMAT ParquetFormat
WITH (
    FORMAT_TYPE = PARQUET
);
```

8.9.5. Basic CETAS Example

Export customer data to **Parquet** in ADLS

```
CREATE EXTERNAL TABLE dbo.Customer_Export
WITH (
    LOCATION = 'customer_export/',
    DATA_SOURCE = MyDataLake,
    FILE_FORMAT = ParquetFormat
)
AS
SELECT CustomerID, FirstName, LastName, Email, City
FROM dbo.Customer
WHERE Country = 'USA';
```

 What happens:

- Synapse runs the **SELECT**.
- Data is written in **parallel** into multiple Parquet files inside **customer_export/**.

- A new external table `dbo.Customer_Export` is created, mapped to those files.

8.9.6. CSV Export

```
CREATE EXTERNAL FILE FORMAT CsvFormat
WITH (
    FORMAT_TYPE = DELIMITEDTEXT,
    FORMAT_OPTIONS (
        FIELD_TERMINATOR = ',',
        STRING_DELIMITER = '"',
        FIRST_ROW = 2 -- skip headers if needed
    )
);

CREATE EXTERNAL TABLE dbo.Sales_Export_CSV
WITH (
    LOCATION = 'sales_csv/',
    DATA_SOURCE = MyDataLake,
    FILE_FORMAT = CsvFormat
)
AS
SELECT SalesID, ProductID, Quantity, TotalAmount
FROM dbo.Sales
WHERE OrderDate >= '2025-01-01';
```

8.9.7. Advanced Example: Partitioned Data

Partition output by **year and month**

```
CREATE EXTERNAL TABLE dbo.Sales_Export_Parquet
WITH (
    LOCATION = 'sales_partitioned/',
    DATA_SOURCE = MyDataLake,
    FILE_FORMAT = ParquetFormat
)
AS
SELECT
    SalesID, ProductID, Quantity, TotalAmount,
    YEAR(OrderDate) AS SalesYear,
    MONTH(OrderDate) AS SalesMonth
FROM dbo.Sales;
```

 The result in Data Lake will be organized into folders like

/sales_partitioned/SalesYear=2025/SalesMonth=01/part-0000.snappy.parquet
/sales_partitioned/SalesYear=2025/SalesMonth=02/part-0001.snappy.parquet

8.9.8. CETAS with Error Handling

You can allow certain % or count of bad rows

```
CREATE EXTERNAL TABLE dbo.Sales_Export_WithTolerance
WITH (
    LOCATION = 'sales_with_tolerance/',
    DATA_SOURCE = MyDataLake,
    FILE_FORMAT = CsvFormat,
    REJECT_TYPE = PERCENT,
    REJECT_VALUE = 5    -- allow up to 5% bad rows
)
AS
SELECT * FROM dbo.Sales;
```

8.10 Creating Views in Azure Synapse & Using Them in Power BI

8.10.1. What is a View in Synapse?

A **view** is a **saved SQL query** that you can treat like a virtual table.

- It doesn't store the data physically (like a table).
- It just stores the **query definition**.
- Every time you query the view, Synapse re-runs the underlying SQL.

👉 Use views when you want to **simplify queries, enforce business logic, or hide complexity**.

8.10.2. Creating Views in Synapse

Views can be created on **Dedicated SQL Pools** or **Serverless SQL Pools**.

Basic View

```
CREATE VIEW dbo.TopCustomers AS
SELECT CustomerID, SUM(TotalAmount) AS TotalSpent
FROM dbo.Sales
GROUP BY CustomerID
HAVING SUM(TotalAmount) > 10000;
```

Now you can simply query:

```
SELECT * FROM dbo.TopCustomers;
```


View on External Data

You can also create views directly on files in Data Lake

```
CREATE VIEW dbo.Sales_2025_View AS
SELECT *
FROM OPENROWSET (
    BULK 'https://mydatalake.dfs.core.windows.net/sales/2025/*.parquet',
    FORMAT = 'PARQUET'
) AS data;
```

This makes **raw files in ADLS** look like a **SQL table** for Power BI or Synapse queries.

Materialized Views

Unlike normal views, a **materialized view** stores the results physically for faster reads.

```
CREATE MATERIALIZED VIEW dbo.MonthlySalesSummary
WITH (DISTRIBUTION = HASH(CustomerID), CLUSTERED COLUMNSTORE INDEX)
AS
SELECT CustomerID, YEAR(OrderDate) AS OrderYear, MONTH(OrderDate) AS OrderMonth,
    SUM(TotalAmount) AS MonthlyTotal
FROM dbo.Sales
GROUP BY CustomerID, YEAR(OrderDate), MONTH(OrderDate);
```

- ✓ Materialized views **auto-refresh** when underlying tables change.
- ✓ Faster for repeated Power BI queries.

8.10.3. Best Practices for Views in Synapse

- Use **schema names** (e.g., `dbo.ViewName`).
- Apply **filters & aggregations** inside the view to reduce query costs.
- Use **materialized views** for frequently used aggregations.
- Avoid `SELECT *` (always specify needed columns).
- Partition or cluster data if working with large datasets.

10.4. Connecting Views to Power BI

Once you create views in Synapse, you can use them directly in **Power BI**.

Steps in Power BI

1. Open **Power BI Desktop**.
2. Go to **Home** → **Get Data** → **Azure** → **Azure Synapse Analytics (SQL Data Warehouse)**.
3. Enter **server name** and **database name** of Synapse.
4. Choose **DirectQuery** (live connection, recommended) or **Import**.
5. In the Navigator window, select your **Views** (they will appear under your schema).
6. Load them into Power BI.

Using the View in Power BI

- The created `dbo.TopCustomers`, it will appear as a table in Power BI.
- can now build a visualization like
 - **Bar chart** → Top 10 customers by total spend.
 - **Time-series chart** → Customer spend over months (if your view has date grouping).